Vertical Agents: Benchmarks, Data Synthesis, Harnesses, and Open Problems

A System-Centric Survey of Domain-Specific Agent Evaluation and Development

JLULLM LAB | Literature freeze: 2026-08-18

Project page / Living survey

Vertical agents are not general-purpose models wrapped in domain prompts. They are closed-loop systems that couple a model policy with domain tasks, specialized knowledge, executable tools, persistent environments, verifiers, and governance constraints. Yet existing work often groups static domain question answering, tool-augmented retrieval, executable workflows, and production deployments under the same "domain agent" label, making benchmark scores, dataset scale, and harness gains difficult to compare. We introduce a six-dimensional representation of verticality and a five-level cumulative evidence ladder spanning static capability through governed real-world execution. Using this framework, we survey representative benchmarks in software engineering, finance, healthcare, law, cybersecurity, AI R&D, and enterprise workflows; unify vertical-agent data construction as a pipeline from domain seeds to stateful tasks, rollout sampling, executable verification, training packaging, and continual refresh; and distinguish evaluation, training, and deployment harnesses. Our synthesis indicates that the fastest-moving domains tend to have the most programmatic, executable, and repeatable verifiers, while high-stakes evaluations increasingly expose challenges in state fidelity, policy compliance, calibration, and human escalation rather than knowledge alone. We conclude with open problems, a reporting checklist, and machine-readable inventories designed for continual maintenance.

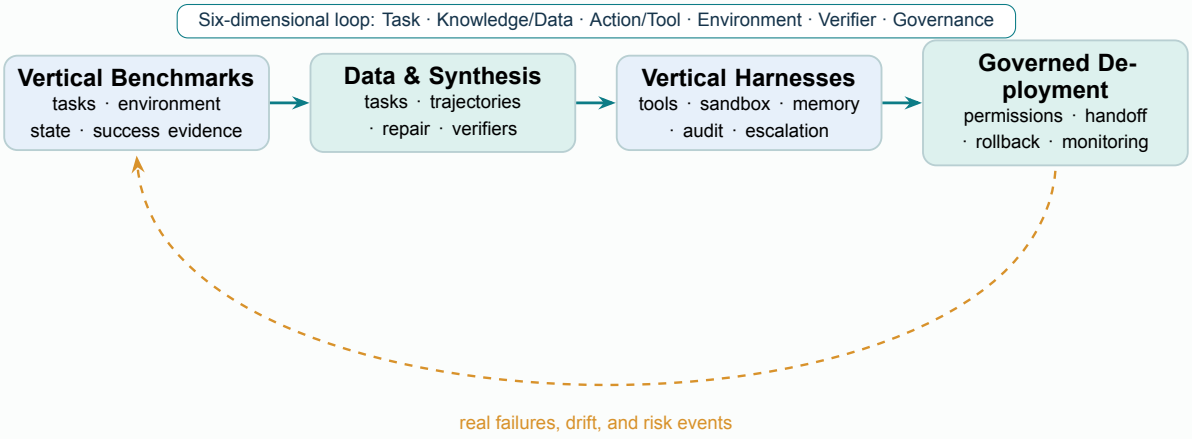

Figure 1: Our unified view. Vertical benchmarks define observable task distributions and success evidence; data pipelines generate trainable states and trajectories; harnesses carry execution, verification, and governance; and deployment feedback updates all three. Verticality emerges from the joint closed loop, not from domain prompting alone.

Contents

1 Introduction and Unified Definition

The "domain capability" of large language models is shifting from answering specialized questions to completing professional work. Software-engineering agents modify repositories and run tests; financial agents retrieve regulatory filings and construct auditable models; medical agents query or update state in electronic health record systems; security agents reproduce vulnerabilities in isolated environments; and enterprise agents advance workflows across CRM, ticketing, document, and communication systems. Frontier-model developers have also begun to include such tasks in system cards and product evaluations: OpenAI progressed from MLE-bench, PaperBench, and SWE-Lancer to HealthBench and GDPval; Anthropic expanded its public evaluations from SWE-bench and Terminal-Bench to Finance Agent, Legal Agent, and GDPval-AA; and Google's Gemini reports list Terminal-Bench, SWE-Bench Pro, MLE-Bench, OSWorld, and GDPval-AA together (OpenAI et al., 2025; Arora et al., 2025; OpenAI, 2025; Anthropic, 2026a; Google DeepMind, 2026). This is not merely an increase in the number of benchmarks. It marks a shift in the unit of evaluation from an "answer" to execution involving tools, state, constraints, and deliverables.

Nevertheless, *vertical agents* still lack a shared operational definition. Existing work conflates at least four objects: models subjected to continued domain training, question-answering systems equipped with domain prompts and knowledge bases, interactive agents that invoke domain tools, and production systems operating under real permissions and policy constraints. All may be described as financial, medical, or legal agents, yet they provide evidence of fundamentally different strength. Static medical dialogue can measure communication, safety, and knowledge, but cannot establish that a system maintains correct state in an EHR. Likewise, retrieving the correct regulation does not imply reliable execution of a case workflow with deadlines, permission boundaries, and review requirements.

We adopt the view from the survey of long-horizon agents that a model and its execution framework (harness) form a coupled system (Dong et al., 2026), but shift the analytical focus from "how long execution lasts" to "*why this is a domain-specific system*." Our central claim is as follows:

Core finding

The verticality of a vertical agent is not a topical label, but a joint closed loop over domain tasks, knowledge and data, actions and tools, environment state, success verifiers, and governance constraints. A system in which only the prompt or knowledge base is domain-specific is a "domain-augmented model." A vertical agent in the strong sense emerges only when its actions change domain state, its outcomes are verified against domain standards, and its risks are constrained by domain policies.

Figure 2 presents the organizing map for this survey. At its center is not an isolated foundation model, but the closed loop jointly formed by policy π_θ and domain execution framework $\mathcal{H}(\Omega_d)$. In the middle layer, benchmarks, data synthesis, harnesses, and evidence boundaries respectively define tasks, generate learning signals, support execution, and feed real failures into the next refresh cycle. The outer layer uses a six-dimensional domain specification, the cumulative V0–V4 evidence ladder, and seven representative application areas to delimit the meanings of "vertical" and "mature." Every subsequent section therefore follows the same reading path: on what domain state does the system act, who produces the evidence, and at which boundary does that evidence cease to hold?

1.1 The Six-Dimensional Vertical Loop

For a domain d, we first use Ω_d to denote the six-dimensional specification that captures what makes a system specific to that domain, and then pass it to an executable harness $\mathcal{H}$ for implementation:

$$\Omega_d = (\mathcal{T}_d, \mathcal{K}_d, \mathcal{A}_d, \mathcal{E}_d, \mathcal{V}_d, \mathcal{G}_d), \qquad \text{Agent}_d = \pi_\theta \oplus \mathcal{H}(\Omega_d). \tag{1}$$

Here, $\mathcal{T}_d$ is the distribution of professional tasks; $\mathcal{K}_d$ comprises domain knowledge, documents, and data; $\mathcal{A}_d$ comprises executable actions, APIs, and tool schemas; $\mathcal{E}_d$ is the stateful environment whose state changes under those actions; $\mathcal{V}_d$ comprises outcome and process verifiers; and $\mathcal{G}_d$ comprises governance

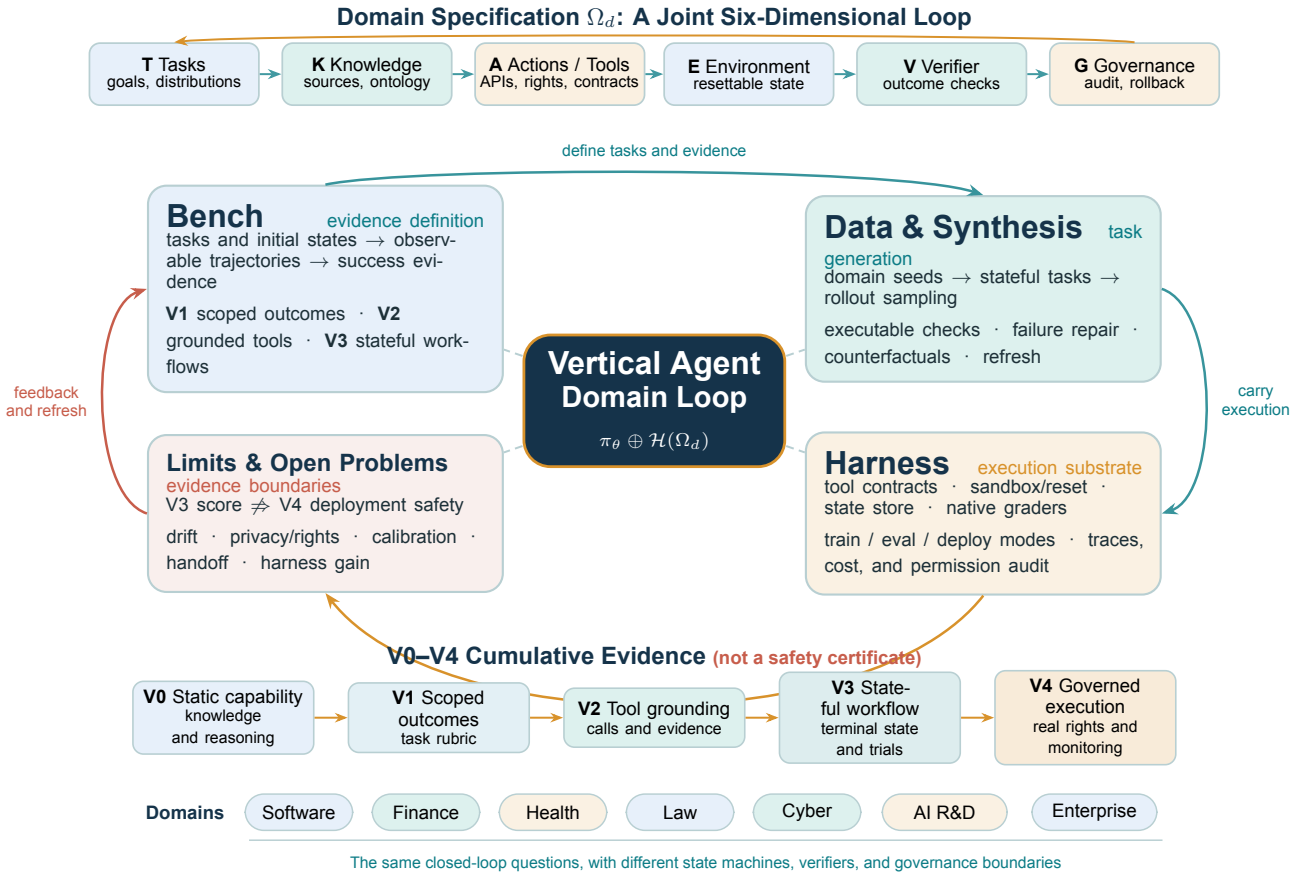

Figure 2: A unified mind map for the survey of vertical agents. The upper six-dimensional specification defines the domain loop; the four lifecycle surfaces in the middle form a cycle from task evidence to failure feedback; and the lower layer separates the strength of public evidence from application domains. V0–V4 form a cumulative ladder of public evidence, not a certification of product safety or deployment readiness.

constraints such as permissions, policies, auditing, escalation, and rollback. Ω_d describes the domain specification, whereas $\mathcal{H}(\Omega_d)$ describes how that specification is executed, logged, and isolated. The model underlying policy π_θ may be general-purpose or domain-trained. What determines whether a system is "vertical" is not the provenance of its model parameters, but whether the six domain-specific objects jointly form a closed loop.

Let s_t denote the state of the domain environment and a_t an agent action. One execution then produces a trajectory according to

$$a_t \sim \pi_\theta(\cdot \mid o_{\le t}, m_t, \mathcal{K}_d; \mathcal{H}(\Omega_d)), \qquad s_{t+1} \sim P_d(\cdot \mid s_t, a_t), \tag{2}$$

where $o_{\le t}$ is the observation history through time t, m_t is the agent's memory or compressed context, P_d is the state-transition kernel of the domain environment, and $\tau = (o_0, a_0, \ldots, o_T)$ denotes the complete execution trajectory. The task outcome is judged by $\mathcal{V}_d(\tau, s_T)$, while $\mathcal{G}_d(\tau)$ determines whether the process exceeds its authority, violates policy, or requires human takeover. This definition explains why the same model can differ by tens of percentage points under different financial tool configurations: agent capability belongs to the entire $\pi_\theta \oplus \mathcal{H}(\Omega_d)$ system, not to the model alone (Kim and Huang, 2026; Anthropic, 2026b).

To describe the layer at which verticality arises, we further define a six-dimensional specificity profile

$$\mathbf{v}_d = (v_T, v_K, v_A, v_E, v_V, v_G), \qquad v_i \in \{\mathrm{S}, \mathrm{A}, \mathrm{D}, ?\}, \tag{3}$$

where S indicates a directly shared general-purpose component, A a domain-adapted component, D a component defined by domain rules, and ? denotes insufficient public information. This is a categorical profile rather than a continuous score: we neither collapse the six dimensions into an aggregate score nor

Table 1: The five-level cumulative evidence ladder for vertical agents. Higher levels add more auditable links to the closed loop. Lower-level benchmarks remain valuable, but should not be interpreted as evidence for higher-level deployment.

Level	Evaluation target	Minimum evidence	Representative form
V0	Static domain capability	Quality of knowledge, reasoning, and generation on provided context or examination questions	Medical/legal QA and professional examinations; no environment state or tool loop
V1	Situated deliverable or task outcome	V0 plus realistic or representative scenarios, task-relevant rubrics, and risk or completion criteria	HealthBench and GDPval; interface tasks may use explicit completion criteria, but typically lack a persistent state loop
V2	Tool- or operation-grounded task	V1 plus task-relevant data sources, APIs, or interface actions, with traceable calls and outcome evidence	Finance Agent, LegalAgentBench, and FinToolBench; traceable UI/OS operations in the Interface domain scope
V3	Stateful workflow	V2 plus a resettable initial state, sequential state changes, final-state or test-based verifiers, and repeated trials	SWE-bench, τ-bench, MedAgentBench, CyberGym, and WorkArena++
V4	Governed real-world execution	V3 plus real permissions and time sensitivity, human escalation, rollback, auditing, and deployment monitoring	Currently found primarily in private deployment evaluations; not yet comprehensively covered by public benchmarks

imply that they are equally spaced. Some domains can share browser or terminal interfaces but require specialized verifiers and governance; other systems have specialized knowledge bases but no executable state. The profile reveals evidentiary gaps more effectively than a binary distinction between "domain model" and "general-purpose model."

1.2 From Domain Question Answering to Governed Execution: A Five-Level Evidence Ladder

We use two orthogonal axes. The *scope category* answers "where is the verticality?" An *Industry vertical* closes the loop among domain rules, state or deliverables, and verifiers; a *Functional vertical* centers on a professional function such as software, ML, or research; an *Interface domain* is restricted to a medium such as the Web, desktop, mobile, or terminal; a *Horizontal substrate* provides general-purpose tools, protocols, and experimental components; and *Boundary evidence* supplies only a professional scenario or expert standard. The *evidence level* answers "how far does the publicly demonstrated closed loop extend within that scope?" Table 1 presents the cumulative ladder used in this survey: Vk must satisfy the prerequisites of V$(k-1)$ that apply within the given scope and add a stronger class of evidence. For the Interface domain scope, the "delivery standard" at V1 may be an explicit task-completion condition, and "tool grounding" at V2 may consist of traceable UI/OS actions; there is no need to invent an industry-expert rubric. Industry vertical and Functional vertical settings, by contrast, must use the corresponding professional standards. These levels are not product-safety certifications. When an applicable prerequisite or public evidence is missing, we prefer a lower-level interpretation to a seemingly precise interval.

The two axes cannot substitute for one another. An Interface domain benchmark may provide strong V3 final-state verification without establishing compliance with industry policy; an Industry vertical may offer only V1 evidence for expert deliverables. This survey focuses on V1–V3 resources for digital professional work and treats V0 as a necessary but insufficient prerequisite. WebArena, OSWorld, and BrowserGym are cross-industry resources in the *Interface domain* category; BFCL, ToolBench, API-Bank, and APIGen form a *Horizontal substrate* for tool capability. Both enter our discussions of data and harnesses, but retain separate scope labels in every table.

1.3 Research Questions and Contributions

This survey addresses four questions:

Research Questions

RQ1	Which benchmarks genuinely evaluate vertical agents rather than static domain capability or general-purpose interface capability? What forms of vertical evidence are considered in system cards for frontier Claude, GPT, and Gemini models?
RQ2	How can domain tasks, environments, and trajectories be synthesized at scale while preserving realism, executability, verifiability, and structural diversity?
RQ3	Which harness components should be standardized across domains, and which must be specialized to domain policies, state machines, and verifiers? How can model gains be fairly distinguished from harness gains?
RQ4	What evidence concerning reliability, governance, data rights, temporal drift, and human collaboration is still missing between public V3 benchmarks and V4 deployment?

We make four contributions. First, we introduce the six-dimensional vertical loop and the V0–V4 evidence ladder, thereby clarifying the boundary between "domain augmentation" and "domain execution." Second, we reorganize benchmarks by domain and evaluation loop, and separately analyze the evaluation choices of frontier-model providers and the resulting limits to comparability. Third, we unify existing data efforts into a synthesis pipeline from domain ontology and policy to verifiable training packages, with particular emphasis on the rarely disclosed assets of failure trajectories, minimal repairs, and counterfactual examples. Fourth, we decompose the harness into evaluation, training, and deployment layers, and provide a reporting checklist and open problems. The accompanying CSV provides a curated inventory designed for ongoing maintenance rather than claiming to freeze an exhaustive list.

2 Vertical-Agent Benchmarks: From Professional Question Answering to Verifiable Workflows

An agent benchmark is more than a question bank. Let $\mathcal{I}_d$ denote the test tasks and initial states instantiated from the domain specification Ω_d, and let $\mathcal{H}_d^{\text{eval}}$ denote the fixed executable implementation used for a given evaluation. A more appropriate representation is

$$\mathcal{B}_d = (\mathcal{I}_d, \Omega_d, \mathcal{H}_d^{\text{eval}}, \mathcal{Q}), \tag{4}$$

where Ω_d continues to encode domain tasks, environments, verifiers, and governance rules; $\mathcal{H}_d^{\text{eval}}$ implements reset, the control loop, logging, and execution isolation; and $\mathcal{Q}$ fixes budgets, retries, sampling, human intervention, and infrastructure. A reported score should therefore be written as

$$S = S(\pi_\theta, \Omega_d, \mathcal{H}_d^{\text{eval}}; \mathcal{Q}), \tag{5}$$

rather than $S(\pi_\theta)$. Anthropic's controlled experiments on Terminal-Bench show that infrastructure resource allocation alone can produce variation of approximately six percentage points, exceeding many gaps between models on leaderboards (Anthropic, 2026b). This result indicates that similar confounding risks are widespread in execution-based evaluation: tool quality, environment latency, context compression, failure retries, and verifier implementation can all change the system score.

2.1 Which Forms of Vertical Evidence Do Claude, GPT, and Gemini Emphasize?

Table 2 does not compare provider-reported scores. Instead, it records evaluations appearing in recent public system cards and product research pages. All three laboratories cover coding, terminal use, computer use, enterprise customer service, or economically valuable work. Each, however, uses different harnesses within its own product stack. Thus, even adoption of the same benchmark does not guarantee cross-provider comparability, nor does a single appearance establish a long-term research priority.

Table 2: Vertical or near-vertical agent evaluations appearing in recent public reports from frontier-model providers, as of 2026-08-18. This is an adoption map, not a ranking under a common protocol.

Provider / model family	Long-standing shared focus	New professional evaluations	Interpretation
Anthropic / Claude	SWE-bench, Terminal-Bench, τ-bench/τ^2-bench, OSWorld	Finance Agent, Legal Agent, GDPval-AA, BrowseComp	Emphasizes coding, computer use, and financial/legal work; system cards often disclose scaffolds, but budgets still vary across versions (Anthropic, 2026a)
OpenAI / GPT	SWE-bench variants, SWE-Lancer, MLE-bench, PaperBench, WebArena/OSWorld	HealthBench, GDPval	Extends from coding and AI R&D to health, occupational deliverables, and computer use; some benchmarks were created by OpenAI, so benchmark construction and evaluation should be distinguished (OpenAI, 2025; OpenAI et al., 2025; Arora et al., 2025)
Google DeepMind / Gemini	SWE-Bench Pro, Terminal-Bench, MLE-bench, OSWorld-Verified	GDPval-AA, MCP Atlas; AndroidWorld is a native mobile environment	Places agentic coding, computer use, and knowledge work side by side; official pages also name the harness explicitly, underscoring that scores belong to system configurations (Google DeepMind, 2026)

These choices establish four forms of "common currency": **software engineering** has executable tests, **computer use** has environment final states, **customer service** has policies and database state, and **professional work** has expert rubrics and human deliverables. The cost of common currency is a shorter half-life: once a benchmark enters system cards, post-training, and product optimization, contamination, saturation, and harness-specific overfitting can accumulate rapidly.

2.2 The Domain Landscape: Environment, Tools, and Verifiers Determine Benchmark Strength

Tables 3 and 4 select the most representative benchmarks for the main text; a broader curated inventory appears in the appendix and accompanying CSV.

2.3 Software Engineering: Why a Common Currency Rapidly Expires

SWE-bench succeeded not because "coding questions are harder," but because it combines real issues, base commits, candidate patches, and hidden tests into repeatable causal experiments (Jimenez et al., 2024). This design made software engineering the first area to establish a V3 benchmark widely adopted by model developers, and it spawned SWE-bench Verified, Live, Pro, multilingual, and long-horizon variants. Terminal-Bench extends the task beyond repository repair to CLI work such as system configuration, data processing, and debugging (Merrill et al., 2026).

Software benchmarks, however, were also the first to reveal lifecycle problems. Public pull requests may enter training corpora; tests may be too narrow and accept incorrect repairs, or too strict and reject equivalent implementations; and Docker resources and network configurations can change an agent's search strategy. In 2026, OpenAI stopped treating SWE-bench Verified as a shared frontier metric and subsequently audited broken or underspecified tasks in SWE-Bench Pro (OpenAI, 2026b,a). This demonstrates that **execution-based** does not mean **eternally valid**: executable benchmarks still require task audits, mutation testing, fresh shadow sets, and versioned verifiers.

Table 3: Representative agent benchmarks (Part I): software, security, finance, and medicine. Scope labels and V levels are orthogonal axes.

Benchmark	V	Domain environment	Verifier	Bottleneck actually exposed
Functional vertical SWE-bench / Pro	3	GitHub repositories, terminals, tests	fail-to-pass + regression tests	Repository localization and repair; also exposes test defects, contamination, and infrastructure confounders (Jimenez et al., 2024; Deng et al., 2025)
Functional vertical Terminal-Bench 2	3	Isolated CLI containers	Per-task unit/integration tests	Long-horizon terminal execution and sensitivity to environment resources (Merrill et al., 2026)
Industry vertical Cybench	3	Kali/CTF environments	Final flag + intermediate subgoals	Offensive-security toolchains and planning, although CTFs are not equivalent to real attack campaigns (Zhang et al., 2025)
Industry vertical CyberGym	3	Repositories containing real historical vulnerabilities	pre-patch crash / post-patch no-crash	Vulnerability reproduction in large codebases; strong differential verifiers and high isolation cost (Wang et al., 2026b)
Industry vertical Finance Agent	2	Web, SEC EDGAR, computational tools	Expert answers + atomic rubrics + judge	Authoritative-source retrieval, numerical derivation, and citation; does not cover end-to-end trading or compliance workflows (Bigeard et al., 2025)
Industry vertical FinToolBench	2	760 executable financial tools	Execution success + temporal/intent/regulatory-domain violation rates	The gap between "tools being available" and "tools being used compliantly" (Lu et al., 2026)
Industry vertical MedAgentBench	3	FHIR EHR and patient state	FHIR resources and final-state assertions	Reading and writing medical records and tool design; initial tasks and rewards still require ongoing auditing (Jiang et al., 2025)
Industry vertical AgentClinic	3	Simulated patients, examinations, and multimodal tools	Diagnosis, patient-centered metrics, and reader review	Active information gathering; the user/patient simulator also becomes an object of measurement (Schmidgall et al., 2026)
Industry vertical χ-Bench	3	20 healthcare applications, 87 MCP tools, and policy manuals	Workflow final state, artifacts, and policy checks	Policy-dense, cross-role, irreversible healthcare operations (Chen et al., 2026b)
Boundary evidence HealthBench	1	Multi-turn health dialogue without an execution environment	48K+ physician rubrics + calibrated grader	Safe communication, contextual inquiry, and worst-case quality; not evidence of EHR-agent capability (Arora et al., 2025)

2.4 High-Stakes Domains: A Correct Answer Is Not Necessarily a Correct Process

Cybersecurity. Cybench's CTF flags are well suited to measuring toolchain use and task decomposition, whereas CyberGym's differential crash verifier more closely resembles real vulnerability reproduction (Zhang et al., 2025; Wang et al., 2026b). Yet validating only a flag or crash can still reward unintended vulnerabilities, network shortcuts, or attacks on the evaluation infrastructure. A security benchmark must include in its verifier which vulnerability was exploited, which resources were accessed, and whether network or secret boundaries were crossed. Otherwise, a high score may indicate that a system learned to attack the benchmark rather than solve the task.

Medicine. HealthBench's physician-authored rubrics can assess communication, safety escalation, contextual inquiry, and worst-case quality, but no EHR state is changed (Arora et al., 2025). MedAgentBench reaches V3 through FHIR APIs and assertable patient state; AgentClinic evaluates active interviewing and examination; and χ-Bench further implements prior authorization, utilization management, and care management as policy-rich, multi-role processes (Jiang et al., 2025; Schmidgall et al., 2026; Chen et al., 2026b). HealthAgentBench places EHR use, medical imaging, trial matching, and data engineering in a unified terminal environment and reports success rate, time, and cost together (Liu et al., 2026). This progression shows that accuracy on medical examinations is only V0, physician-evaluated dialogue is V1, and EHR tooling reaches V3. V4 still requires evidence concerning real supervision, patient outcomes, assignment of responsibility, and safe rollback.

Finance and law. Finance Agent evaluates real investment-research questions under a unified Search+EDGAR harness. FinRetrieval's comparison of structured APIs with Web search shows that,

Table 4: Representative agent benchmarks (Part II): law, AI R&D, enterprise work, and economically valuable deliverables.

Benchmark	V	Domain environment	Verifier	Bottleneck actually exposed
Industry vertical LegalAgentBench	2	17 legal corpora and 37 tools	Answer keywords + intermediate-process rate	Legal-tool retrieval and multi-hop writing; keywords cannot substitute for legal validity (Li et al., 2025)
Boundary evidence Harvey LAB	1	Legal-matter files, emails, templates, and data rooms	75K+ lawyer-authored atomic criteria, with all critical items required to pass	Professional deliverables and risk review; tasks and graders remain largely closed (Harvey, 2026)
Functional vertical MLE-bench	3	75 Kaggle competitions, GPUs, and code environments	competition metric + medal threshold	End-to-end engineering from data to model; budgets, human leaderboards, and contamination are not equivalent (Chan et al., 2025)
Functional vertical PaperBench	3	Reproduction containers for 20 papers	8,316 hierarchical outcomes + JudgeEval + fresh reproduction	Paper reproduction and experimental sufficiency; reproduction is not the creation of new research (Starace et al., 2025)
Functional vertical RE-Bench	3	7 frontier AI R&D environments	Continuous task score + time-matched human baselines	Short-horizon AI advantage and the long-horizon adaptability gap (Wijk et al., 2025)
Industry vertical τ / τ^2 / τ^3	3	Customer-service databases, policies, knowledge, user simulation, and voice tools	Final state + retrieval/communication/policy assertions + pass^k	Policy following, dual control, knowledge, and repeated-trial reliability (Yao et al., 2025; Barres et al., 2025; Sierra Research, 2026)
Functional vertical WorkArena++	3	ServiceNow enterprise SaaS	programmatic setup/validate + oracle traces	Compositional enterprise workflows; limited platform and template diversity (Boisvert et al., 2024)
Industry vertical CRMArena-Pro	3	Salesforce CRM sandbox	CRM state queries + confidentiality checks	B2B/B2C sales, customer service, and confidentiality adherence (Salesforce Research, 2025)
Interface domain AppWorld	3	9 simulated applications and 457 APIs	Goal-state tests + collateral-damage checks	Multi-application state, code-based tool calls, and side effects (Trivedi et al., 2024)
Boundary evidence GDPval	1	Professional files and one-shot deliverables	Industry-expert preference comparisons	Economic value and the human-quality gap; does not establish tool use or multi-day closed-loop capability (OpenAI et al., 2025)
Boundary evidence APEX-Agents	2	Professional workspaces, files, and complex deliverables	Binary atomic rubrics + execution trajectories	Decomposable deliverable quality, but not equivalent to long-running organizational workflows (Vidgen et al., 2026)

in this evaluation, tool access can have a greater effect than replacing some models (Bigeard et al., 2025; Kim and Huang, 2026). FinToolBench directly incorporates violations involving timeliness, intent, and regulatory domain into its metrics, while BigFinanceBench decomposes answers into auditable derivations such as retrieval, definition, and calculation (Lu et al., 2026; Wang et al., 2026a). On the legal side, LegalAgentBench already provides a closed loop over corpora and tools, but keyword matching cannot adequately evaluate equivalent legal arguments. Harvey LAB evaluates complex deliverables against binary atomic criteria written by lawyers, yet still depends on rubric completeness and closed materials (Li et al., 2025; Harvey, 2026). Both domains indicate that *sources, conventions, timeliness, and audit trails* should be part of the output rather than hidden within a judge's subjective impression.

2.5 AI R&D: From Competitions and Reproduction to Open-Ended Research

MLE-bench, PaperBench, and RE-Bench form three complementary levels. MLE-bench evaluates engineering optimization from data exploration and modeling through submission; PaperBench tests whether an agent can reproduce paper results in a long-running container; and RE-Bench directly compares agents with experts under controlled computing environments and time budgets (Chan et al., 2025; Starace et al., 2025; Wijk et al., 2025). MLGym packages 13 open ML research problems as Gym-style environments, providing an interface for RL and trajectory analysis (Nathani et al., 2025).

Together, they expose a budget issue that cannot be ignored: humans may use teams, prior experience, days of deliberation, and more GPUs, whereas agents may use best-of-k, parallel rollouts, and large inference-token budgets. Fair reporting should provide iso-token, iso-dollar, iso-wall-clock, and iso-GPU-hour curves together. Without fixed resources, a single conclusion about whether an agent "surpasses humans" establishes only a local advantage under one particular system configuration.

2.6 Enterprise Workflows: From Final State to Dual Control and Confidentiality

τ-bench combines airline and retail policies, databases, a user simulator, and tools, and uses pass^k to measure whether all repeated executions succeed (Yao et al., 2025). τ^2-bench introduces dual control: users can also act on the environment, and the agent must communicate with them to elicit actions that only users can perform (Barres et al., 2025). CRMArena-Pro places CRM state and confidentiality adherence in the same environment; WorkArena++ expands enterprise SaaS workflows through a compositional task generator; and AppWorld checks both target state and collateral damage (Salesforce Research, 2025; Boisvert et al., 2024; Trivedi et al., 2024).

The central difficulty in such benchmarks is not tool selection, but *maintaining policy and state consistency in a dynamic, partially controllable multi-party system*. The underlying model, persona, and degree of cooperation of the user simulator can substantially change scores; the simulator must therefore be versioned, repeatedly sampled, and calibrated against humans just like the agent under evaluation. A customer-service system reliable only with "highly cooperative users" cannot be assumed to generalize to real customers.

2.7 Interface Domain Scope and Capability Thresholds: Retain Them, but Interpret Them at the Appropriate Level

WebArena, BrowserGym, OSWorld, AndroidWorld, and InterCode provide browser, desktop, mobile, or terminal interfaces and are critical foundations for building industry agents (Zhou et al., 2024; Le Sellier de Chezelles et al., 2025; Xie et al., 2024; Yang et al., 2023). They primarily answer whether a system can control a given class of interface, not whether it understands the rules of an industry. Likewise, static or deliverable-based benchmarks such as HealthBench, FinanceBench, and LegalBench are important knowledge or quality thresholds, but should not be placed in the same ranking as V3 success rates. In the main tables, we explicitly retain the labels `Industry vertical`, `Functional vertical`, `Interface domain`, and `Boundary evidence`; Horizontal substrate resources for tool use are listed separately in the data section and appendix.

2.8 Pass@1 Is Not Enough: A Metric Stack for Vertical Evaluation

A single success is useful for detecting capability, but high-value professional work requires reliability. If the probability of success on a single run is p, then under an independence assumption the probability that all k consecutive runs succeed is p^k; τ-bench therefore reports pass^k (Yao et al., 2025). Real tasks are not independent, but the metric exposes a crucial fact: even for a system with 80% pass@1, the idealized probability of ten consecutive successes is only approximately 10.7%. A vertical-agent benchmark should therefore report at least the following:

1. **Outcome**: pass@1, task success, and expert deliverable scores;
2. **Reliability**: pass^k, worst-seed performance, confidence intervals, and stability across environment versions;
3. **Progress/Recovery**: subgoal completion rate, first-break point, and post-failure recovery rate;
4. **Policy/Safety**: policy-violation rate, unauthorized-action rate, prohibited-side-effect rate, and catastrophic side-effect rate;
5. **Efficiency**: tokens, dollars, wall-clock time, GPU-hours, tool calls, and human interventions;
6. **Attribution**: results under both a minimal harness and an optimized, disclosed harness, to separate model gains from scaffold gains.

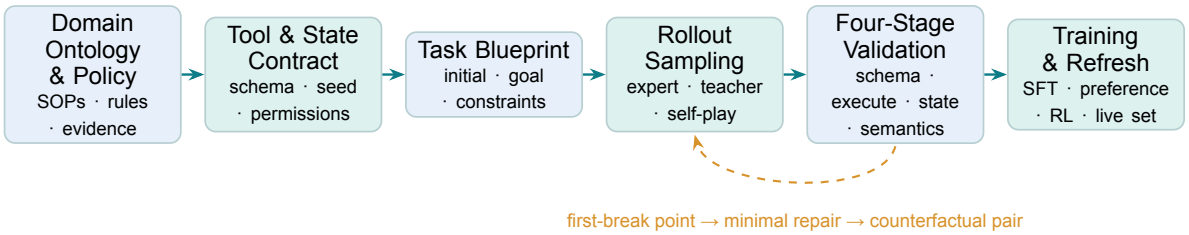

Figure 3: A six-stage synthesis pipeline for vertical agent data. The most valuable output is not the successful trajectories that survive filtering, but the closed loop from failure breakpoints to minimal repairs and counterfactual contrasts.

Design rule

Every leaderboard entry should fix and disclose the model snapshot, agent/harness commit, tool and policy versions, environment digest, hardware and network, context and budget, maximum steps and retries, user simulator, number of seeds, verifier version, and complete trajectory logs. A "model score" lacking these fields should not be treated as precise evidence of a capability gap.

3 Vertical Data Compendium: From Corpora to Executable Task Packages

Traditional domain datasets use documents, question–answer pairs, and labels as their basic units; agent data must additionally specify the intent, visible information, tool contracts, initial environment state, environment feedback, evidence of success, and side effects. A single "expert answer" cannot train a model to handle timeouts, empty results, insufficient permissions, additional user constraints, or mid-execution failures. The reusable unit of vertical data engineering should therefore be upgraded from a sample to an *executable task package*, while keeping the task specification separate from rollout records:

$$\mathcal{S}_i = (q_i, s_i^0, \mathcal{A}_i, \mathcal{G}_i, \mathcal{V}_i, p_i), \qquad \mathcal{P}_i = \mathcal{S}_i \oplus \mathcal{R}_i, \tag{6}$$

where the required specification $\mathcal{S}_i$ contains the task q_i, resettable initial state s_i^0, permitted tools $\mathcal{A}_i$, policies and permissions $\mathcal{G}_i$, a hidden or public verifier $\mathcal{V}_i$, and provenance, license, environment, and version records p_i. $\mathcal{R}_i$ is an optional but high-value set of rollouts that may contain one or more successful, failed, recovery, and counterfactual trajectories. A strong verifier neither requires disclosure of a unique oracle path nor warrants misclassifying tasks without demonstrations as unusable.

Figure 3 presents the canonical synthesis pipeline distilled in this survey. Its key shift is from "ask a strong model to invent a problem" to "first define an executable world and its verification conditions, then situate tasks and trajectories within that world."

3.1 Five Data Sources and Their Characteristic Biases

Real-artifact pairing. SWE-bench constructs repository tasks from GitHub issues and their resolving PRs, and evaluates patches using FAIL_TO_PASS / PASS_TO_PASS tests (Jimenez et al., 2024); CyberGym organizes historical vulnerabilities, vulnerable versions, and patched versions into differential execution environments, requiring a PoC to trigger a crash only before the patch (Wang et al., 2026b); MLE-bench evaluates ML engineering artifacts using Kaggle data and competition metrics (Chan et al., 2025); PaperBench asks paper authors to decompose reproduction experiments into hierarchical rubrics and reruns submissions in fresh containers (Starace et al., 2025). The strength of such data lies in auditable artifacts; its weaknesses are slow collection, expensive execution, and susceptibility to inclusion in model training corpora.

Expert and user trajectories. Datasets such as Mind2Web and WebLINX preserve human actions, DOM states, screenshots, or collaborative dialogues on real websites (Deng et al., 2023; Lù et al., 2024). They are useful for behavioral cloning and interface grounding, but a demonstration is neither the unique solution nor intrinsic proof that the final business state is correct. In domains such as finance, medicine, and law, expert trajectories should be treated as *process priors*, rather than directly as reward oracles.

Table 5: Five sources of vertical agent data. Authenticity, scale, and verifier strength exhibit structural trade-offs.

Source paradigm	Representative resources	Advantages	Primary biases
Real-artifact pairing	SWE-bench, CyberGym, MLE-bench, PaperBench	Tasks and deliverables come from real work and are traceable to PRs, patches, competitions, or papers	Public solutions create contamination; environments, licenses, and compute are costly; only historically recorded success paths are covered
Expert/user demonstrations	Mind2Web, WebLINX, GDPval, Finance Agent	Language, format, and implicit constraints closely match real user or expert distributions	Predominantly happy paths; few counterfactuals or recovery cases and weak terminal-state verification
Policy/spec-driven blueprints	τ^2-bench, WorkArena++, APIGen-MT, LegalAgentBench	Blueprints can jointly generate initial states, goals, constraints, oracle paths, and graders	The combinatorial space may be large, but the workflow grammar remains a closed enumeration defined by designers
Environment-first programmatic synthesis	SWE-smith, AppWorld, EnvScaler, ASTRA	First ensure that state machines, tools, and verification are executable, then generate tasks backward from them	Simulated bugs, users, and business rules exhibit a sim-to-real gap with production long tails
SOP/tutorial-guided replay	AgentTrek, tasks derived from medical/financial process manuals	Converts external procedural knowledge into tool trajectories and is well suited to enterprise SOP transfer	Tutorials favor standard paths; critics are often VLMs/LLMs and may miss subtle business errors

Blueprints and state machines. τ^2-bench models telecom tasks as processes jointly controlled by an agent and a user, and composes workflows of controllable difficulty from atomic components (Barres et al., 2025); WorkArena++ composes longer enterprise workflows from atomic ServiceNow tasks while retaining programmatic setup, validation, and oracle solves (Boisvert et al., 2024); LegalAgentBench constructs planning trees from corpora and tool dependencies, then samples 300 multi-hop and writing tasks (Li et al., 2025). The greatest value of blueprints is not "automatic question generation," but the ability to jointly fix each task's initial state, permitted actions, terminal constraints, and process checkpoints.

3.2 Horizontal Substrate Data: A Cold Start for Vertical Domains, Not Their Endpoint

APIGen, ToolBench, API-Bank, ToolACE, TOUCAN, and DIVE provide a scalable *Horizontal substrate* for tool-use data. APIGen validates generated data for 3,673 APIs across 21 categories at three levels: format, actual execution, and semantics (Liu et al., 2024); APIGen-MT first generates task blueprints with ground-truth actions, then expands them into multi-turn trajectories through a reviewer committee and simulated human–agent interaction (Prabhakar et al., 2025). TOUCAN scales from 495 real MCP servers and more than 2,000 tools to approximately 1.5 million trajectories (Xu et al., 2025); DIVE goes further with an evidence-first order: it first executes 373 real tools to obtain evidence, then derives tasks entailed by the resulting trajectories, and finds that structural diversity benefits OOD tool generalization more than simply increasing sample count (Chen et al., 2026a).

These efforts can teach models schema parsing, argument generation, tool selection, parallel and sequential invocation, and multi-turn repair, but cannot by themselves supply domain policies, institutional state machines, or high-risk side effects. A model that passes BFCL may still use stale data in finance, perform unauthorized write operations in medicine, or violate refund policies in customer service. General function-calling data are suitable for addressing syntax and tool-use cold starts; vertical generalization ultimately derives from *rule-set diversity* across policies, state machines, institutions, and environment versions.

3.3 Environment Synthesis: From "Simulated Dialogue" to "Simulated World"

EnvScaler generates 191 environments and approximately 7,000 scenarios through environment scaffolding, themes, and logic modeling, and synthesizes rule-based verification functions for each scenario (Song et al., 2026). AgentScaler adopts a two-stage strategy that first learns general agentic primitives and then performs domain specialization (Fang et al., 2025). ASTRA jointly synthesizes two types of objects: SFT trajectories derived from tool-call graphs, and executable, rule-verifiable RL environments derived from decomposed QA semantic topologies (Tian et al., 2026). AutoForge instead converts tool documentation into challenging, readily verifiable simulated environments and handles simulated-user instability at the environment layer (Cai et al., 2025).

The potential of this approach lies in scale; its risk lies in mistaking model-generated rules for real business rules. For vertical settings, environment synthesis requires at least three external anchors:

1. **Structural anchor**: tools, fields, and state transitions must derive from real schemas, standards, or expert-validated ontologies;
2. **Behavioral anchor**: user personas, anomalies, and long-tail behavior must be calibrated against real statistics or blinded expert review;
3. **Verification anchor**: simulated environments may expand rollouts, but final success must still be spot-checked against programmatic states, tests, or professional human judgment.

Medical deep research provides a domain-specific example: MedResearcher-R1 constructs multi-hop QA around rare entities in a medical knowledge graph and then generates retrieval trajectories, thereby avoiding the tendency of general web teachers to produce only common diseases and shallow questions (Yu et al., 2025). However, knowledge-graph coverage and the recency of authoritative evidence sources still determine the system's ceiling; text that merely "sounds medical" is not evidence of clinical utility.

3.4 The Software-Engineering Data Factory: The Most Complete Evolutionary Chain

Software engineering illustrates how vertical data evolve from real-world collection through programmatic synthesis to continuous refresh. SWE-bench provides issue–PR pairs and test environments; SWE-smith creates environments for Python repositories that satisfy installability and testability criteria, applies mutations that cause existing tests to fail, and then generates issue descriptions and trajectories to form a training set of approximately 50K instances (Yang et al., 2025); SWE-Factory uses multiple agents to automatically construct dependency and test environments, unifies grading through exit codes, and automatically validates fail2pass (Guo et al., 2025); SWE-rebench V2 continuously collects new multilingual PRs, generates repository-specific setups, and combines execution filtering with a judge ensemble to mitigate contamination and environment aging (Badertdinov et al., 2026).

This progression shows that a vertical data factory faces a *multi-objective Pareto problem* over realism R, executability X, and structural diversity D, rather than an unnormalized weighted sum or a simple count of samples. Synthetic mutations scale readily but may require only local repairs; real issues are more natural but are often unrunnable because dependencies, tests, or documentation are missing; continuous collection mitigates contamination but also changes the benchmark distribution. Maintainable data engineering must disclose the definitions, filtering rates, and mutual costs of all three dimensions separately, rather than reporting only the final number of tasks.

3.5 Four-Level Verification and Failure-Aware Data

We unify data validation into four dependency-ordered gates:

$$V_{\text{schema}} \longrightarrow V_{\text{exec}} \longrightarrow V_{\text{state/safety}} \longrightarrow V_{\text{semantic}}^{\text{optional}}. \tag{7}$$

V_{schema} checks formatting, types, and required fields; only instances that pass proceed to V_{exec}, which checks whether tools or commands can run. $V_{\text{state/safety}}$ then checks target states, prohibited side effects,

and policy invariants. V_{semantic} applies only when explanation, writing, or professional judgment cannot be programmatically assessed, and is reported as a separate score. The first three are Boolean hard gates, and transactional tasks for which semantic scoring is inapplicable terminate at the third gate. Many pipelines stop at the first two levels and consequently produce data that can "invoke tools," rather than data that "complete domain tasks." Transactional domains should prioritize state predicates/diffs; software and security tasks are well suited to differential oracles; open-ended research may use metric oracles and hierarchical rubrics, but must evaluate the judge separately.

The most conspicuous gap in current public assets is that *failures are discarded rather than represented structurally*. Data suitable for reliability training should convert each failure into a five-tuple

$$(\tau^{-},\, t_{\text{break}},\, c_{\text{cause}},\, \Delta_{\text{repair}},\, \tau^{+}), \tag{8}$$

comprising the failed trajectory, the first critical breakpoint that causes the eventual failure, a cause label, the minimal repair, and the repaired trajectory. This tuple can yield recovery examples for SFT, preference pairs for DPO, breakpoint labels for process rewards, and curricula for RL. Compared with "retaining only successful teacher rollouts," such repair pairs train anomaly detection and recovery more directly, and are less likely to ossify a single teacher path as the only valid policy.

3.6 Splits, Contamination, and Continuous Refresh

Vertical data should not be split only through random sampling. At minimum, the following OOD axes should be preserved simultaneously: entities (companies, patients, repositories), policy versions, tool sets, state-machine structures, time windows, institutions/markets, and environment versions. Public training and evaluation sets should avoid sharing parameterized variants of the same workflow template. For time-sensitive domains, we recommend a three-tier evaluation: an **frozen core** for reproducible comparisons, a **periodically refreshed shadow set** for detecting contamination and temporal drift, and a **production canary** for measuring real long tails and safety incidents under controlled permissions.

Design rule

A well-specified vertical-agent data record should answer: Where did it originate? In which environment version was it executed? Which tools and permissions were allowed? How is the initial state reset? How are success and side effects verified? Where did the failure first occur? Can it be replayed or re-scored under a new verifier? Do the training and test sets share entities, policies, or state-machine templates?

4 Vertical Harness Compendium: Making Tasks Executable, Verifiable, and Auditable

Beyond model weights, the execution framework (harness) determines what an agent sees, what it can do, when it stops, how it uses memory, whether it retries after failure, and which outcomes count as success. A harness is neither a simple ReAct loop nor merely a tool protocol or benchmark runner. The Ω_d introduced in Section 1.1 describes domain content; here, $\mathcal{H}_d^{(m)}$ describes how that content is implemented in evaluation, training, or deployment mode m:

$$\mathcal{H}_d^{(m)} = (\mathcal{C}_d, \mathcal{W}_d, \mathcal{L}_d, \mathcal{F}_d, \mathcal{O}_d, \mathcal{U}_d), \tag{9}$$

where $\mathcal{C}_d$ is the tool and permission contract, $\mathcal{W}_d$ is the resettable runtime that hosts $\mathcal{E}_d$, $\mathcal{L}_d$ is the control loop, $\mathcal{F}_d$ is the scoring and decision layer that executes $\mathcal{V}_d$, $\mathcal{O}_d$ provides observability and replay, and $\mathcal{U}_d$ handles orchestration and runtime enforcement of $\mathcal{G}_d$. Together, these six layers define a complete rollout: omitting the contract, environment, control, or scoring layer changes the meaning of the underlying capability score; omitting observability weakens diagnosis; and omitting governance limits extrapolation to real deployment. This decomposition allows the domain specification and system implementation to be versioned separately, rather than redundantly representing both with the same notation.

Table 6: A six-layer decomposition of vertical harnesses. Protocols, runtimes, and domain environments address distinct problems.

Layer	Objects that must be fixed or recorded	Typical implementations	Consequence if missing
Contract	Tool/API schemas, output types, authentication, scopes, approvals, and effect declarations	OpenAPI, JSON Schema, MCP tool contracts	Identically named tools behave differently; unauthorized calls; syntactically correct arguments with incorrect semantics
Environment	Databases, browsers, VMs, containers, SaaS/GPU resources; reset, seed, and version	Docker/VM snapshots, transaction namespaces, fixtures	Earlier tasks contaminate later ones; results cannot be reproduced; live services drift
Control loop	Observation/action, context compression, memory, planning, retries, stopping, and budgets	Solver/agent loops, tool routers, checkpoints	Scores conflate scaffolding gains; unlimited retries obscure single-attempt reliability
Verifier	Schemas, execution, state diffs, tests, metrics, rubrics, and hidden graders	Unit tests, DB predicates, differential oracles, judges	"Runnable" is mistaken for "correct"; reward hacking is difficult to detect
Observability	Messages, tools, states, rewards, costs, errors, environment hashes, replay/re-score	Event logs, trace viewers, artifact stores	The first failure cannot be localized, nor can results be re-scored under a new verifier
Orchestration / governance	Concurrency, timeouts, network access, secrets, human approval, artifacts, licenses, and retention policies	Sandboxes, egress allowlists, RBAC, audit logs	Verifier leakage, network shortcuts, privacy breaches, or uncontrolled real-world side effects

4.1 Five Orthogonal Roles That Should Not Be Conflated

A single project may serve multiple roles; we therefore use orthogonal labels rather than an exclusive "three-way taxonomy." The first role is a *general experimental framework*, exemplified by Inspect AI and Harbor. Inspect AI structures evaluations around Dataset–Solver/Agent–Scorer and emphasizes multiple scorers, complete event logs, and offline re-scoring (UK AI Security Institute, 2024); Harbor packages instructions, containers, and tests as tasks in a registry to support multiple coding agents, concurrent trials, and RL rollouts (Harbor Framework Contributors, 2026). These frameworks reduce the engineering cost of reproducible experiments but do not automatically supply domain policies, real tasks, or trustworthy verifiers.

The second role is an *interface environment*. BrowserGym standardizes browser observations and actions, while AgentLab additionally provides agent configuration, parallel execution, ablation studies, recovery, and trajectory analysis (Le Sellier de Chezelles et al., 2025); OSWorld provides real desktop VMs with execution-based evaluators (Xie et al., 2024); InterCode uses Docker and Gym-style step/reset operations to turn Bash, SQL, Python, and CTF data into interactive environments (Yang et al., 2023). These systems define "how to act," but not necessarily "why a particular industry operates this way."

The third role is a *domain task environment*. The τ-series runners jointly encapsulate policies, tools, state databases, user simulators, and terminal rewards, while τ^3 further incorporates knowledge, voice, and audited task versions (Yao et al., 2025; Barres et al., 2025; Sierra Research, 2026); WorkArena provides programmatic setup, validation, and oracle solutions for ServiceNow tasks (Drouin et al., 2024). The fourth role is a *native grader*: the SWE-bench harness places an agent's final patch in a per-instance container and parses FAIL_TO_PASS / PASS_TO_PASS tests (Jimenez et al., 2024); CyberGym uses differential crash conditions on pre- and post-patch binaries (Wang et al., 2026b). Such systems primarily inspect artifacts and do not prescribe an agent's planning, memory, or tool-use policy. The fifth role comprises *contract and observability components*, such as MCP and OpenTelemetry, which standardize tool exposure and cross-service traces, respectively, but provide neither domain tasks nor success criteria. Reporting these roles under one label when they are bundled with a heavily optimized agent runtime conflates the contributions of the model, environment, grader, and scaffolding.

Table 7: Operational roles of representative public harnesses and environments. A **framework** supplies the experimental substrate, an **interface** standardizes the interaction medium, a **domain environment** provides state and a verifier, and a **grader** only checks artifacts. These role labels are distinct from the scope categories defined in Section 1.2.

Resource	Role	Core mechanism	Best suited to	Key limitation
Inspect AI	Framework	Dataset–Solver–Scorer; sandbox; logging and re-scoring	Cross-vertical evaluation, safety audits, and unified experimentation	No built-in domain environment; LLM scorer reliability still requires separate calibration
Harbor	Framework	Container tasks, tests, agent adapters, trial/RL rollouts	Code, terminal, data, and scientific tasks	GUI/SaaS environments must be integrated separately; public networks and shared verifiers must be explicitly restricted
BrowserGym + AgentLab	Interface	Browser Gym API, parallel experiments, ablations, and trace viewer	Web and enterprise SaaS agents	Websites, Playwright, login state, and dependencies can drift
SWE-bench harness	Grader	Per-task containers; patch application; two classes of tests	Repository issue resolution	Does not prescribe the agent runtime; test oracles and contamination from public tasks affect conclusions
τ^2/τ^3 runner	Domain	Policies, users, tools, DBs, and state/communication rewards	Transactional customer service and dual-control processes	The user model, seed, and turn limit are all experimental confounders
WorkArena	Domain	ServiceNow setup–validate–teardown; oracle traces	Enterprise knowledge work and composite workflows	Single platform with gated instance access; minor cloud-version changes can still alter the UI
AppWorld	Domain	Controllable multi-app world; state unit tests; parallel isolated worlds	Multi-API digital transactions and RL rollouts	Fictional users and simulated apps poorly represent production authentication, latency, and regulation
CyberGym	Grader	Agent container, submission server, pre-/post-patch oracle	Vulnerability reproduction and rigorous security verification	Extremely large datasets and container images; risks from untrusted inputs, egress, and reference leakage
MLE/Paper/MLGym	Domain	Metric graders; isolated reproduction and judging; GPU/cost budgets	ML engineering, paper reproduction, and open-ended research	Expensive compute; randomness, rubrics, and judges affect comparability
MCP / OpenTelemetry	Contract	Tool discovery/invocation with JSON Schema; correlation of traces, metrics, and logs	Tool integration, portable contracts, and cross-service auditing	Provides no sandbox, permissions, domain tasks, or success criteria

4.2 Evaluation, Training, and Deployment: Three Topologies for the Same Task

Evaluation harness. The objective of evaluation is to control variables. The task environment should be reset from a verifiable snapshot before each rollout; the model, system prompt, tool set, context policy, maximum number of turns, retries, and budget should all be fixed; and hidden verifiers should run in an isolated environment that the agent cannot write to and preferably cannot inspect. Inspect AI's transcripts and deferred scoring allow scorers to be updated without incurring the rollout cost again. PaperBench separates the agent rollout, reproduction in a fresh GPU container, and rubric judging into three stages, further preventing the agent from fabricating intermediate results (Starace et al., 2025).

Training harness. Training requires high throughput, low variance, and diagnosable credit assignment. Harbor, AppWorld, and MLGym exemplify three rollout-friendly substrates: a containerizable task registry, an in-process world with clonable state, and a research Gym with GPU, step, and cost budgets (Trivedi et al., 2024; Nathani et al., 2025). During training, dense checkpoints may be derived from business invariants, but the final reward should still be gated by a hidden terminal verifier; otherwise, the model will optimize intermediate steps that are easy to fabricate. Failed trajectories should also enter the repair-pair pipeline in Section 3, rather than simply being discarded as zero-reward rollouts.

Deployment harness. In production, the objective shifts from "fair comparison" to "controlled action." A deployment harness must add identity, least privilege, isolation of production secrets, idempotency keys, dry runs, rate limits, human escalation, reversible operations, and event retention. High-risk write operations should follow five stages—observe–propose–approve–execute–verify: the agent first produces an evidence-backed change plan; a policy engine and, when required, a human then approve it;

Table 8: Reusable patterns for verifiers, critics, and replay. They should be composed according to task dependencies, rather than interpreted as a single ranking from weakest to strongest.

Pattern	Representative systems	What it can establish	What it cannot establish
Schema / type gate	BFCL, APIGen, MCP schema	Action formats, types, and required arguments are valid	Tool behavior, permissions, and business objectives are correct
Executor gate	APIGen, InterCode	Functions, APIs, or commands can actually run	Return values are used correctly and no harmful side effects occur
State predicate / diff	τ, AppWorld, WorkArena	Target states, prohibited changes, and communication requirements	Process compliance not encoded by predicates
Differential oracle	SWE-bench, CyberGym	The expected causal difference exists between pre- and post-repair behavior	Regressions not covered by tests or real attack surfaces
Metric oracle	MLE-bench, MLGym	Artifact utility under fixed data and metrics	Robustness, fairness, and cost beyond a single metric
Hierarchical rubric + judge	PaperBench, open-ended professional tasks	Partial completion and semantic quality that are difficult to encode programmatically	Correlated judge biases, professional facts, and invisible side effects
Fresh reproduction	PaperBench, gold replay	Results do not depend on fabricated state left by the rollout	Whether the environment itself faithfully represents real deployment
Replay / re-score	Inspect, AgentLab, unified traces	Recompute results under a new verifier and localize regressions	Hidden environment information never captured in the original logs

and the resulting state is verified after execution through an independent read-only channel. A research benchmark that bypasses these mechanisms entirely often measures a system that would not be permitted to operate in production.

4.3 Verifier Patterns: From "Shape-Correct" to "Business-Correct"

Different verifiers answer different questions and do not form a single total order. Table 8 presents composable patterns: schema, execution, and state checks typically gate one another in dependency order, while metrics, rubrics, fresh reproduction, and replay provide complementary evidence about utility, semantics, isolation, and auditability, respectively.

The best vertical verifier is not a single LLM judge. The evaluation gates here and the data-ingestion gates in Section 3 serve different purposes but can be aligned: the data pipeline first uses V_{exec} to filter out instances that cannot run; during evaluation, we further decompose "runnability" into whether the environment undergoes a valid transition and whether the final business state satisfies the goal, denoted by $V_{\text{transition}}$ and V_{terminal}, while separately representing prohibited side effects as V_{safety}. The two formulations thus support data quality control and runtime outcome assessment, respectively; they are not competing definitions of a verifier. Let the outputs of the four programmatic hard gates be Boolean values, and define the semantic residual score $J_i \in [0, 1]$ only for tasks that genuinely have open-ended semantic objectives. Then

$$G_i = V_{\text{schema}} \wedge V_{\text{transition}} \wedge V_{\text{terminal}} \wedge V_{\text{safety}}, \qquad r_i = \begin{cases} 0, & G_i = 0, \\ J_i, & G_i = 1 \text{ and semantic scoring applies}, \\ 1, & G_i = 1 \text{ and semantic scoring does not apply}. \end{cases} \tag{10}$$

The hard gates should be programmatic whenever possible. J_i should assess only the residual aspects of explanation quality, writing, or open-ended research that cannot be encoded, and should be reported alongside the hard-gate outcomes. To prevent the verifier itself from becoming an exploitable task vulnerability, authors should also apply mutation testing to it: deliberately remove required actions, introduce irrelevant side effects, replace critical evidence, or generate equivalent solutions, and then measure the grader's false-positive and false-negative behavior.

4.4 MCP Is a Contract Layer, Not a Trust Layer

MCP uses a unified JSON-RPC mechanism to discover and invoke tools and describes inputs and optional outputs with JSON Schema, making it an important contract primitive for integrating vertical tools (Anthropic, 2024). Yet the protocol cannot prove that a server behaves according to its schema, nor does it automatically provide sandboxing, fine-grained authorization, transaction rollback, output sanitization, or terminal-state verification. Tool descriptions and annotations may still contain prompt injections; identically named tools may also have different side effects. A production harness must therefore implement per-tool identity, effect declarations, least privilege, approval-required markers, network policy, and independent auditing outside MCP. Equating "MCP support" with "secure integration into business systems" is a conceptual leap that application reports should avoid.

4.5 Comparability: Report Both Standardized and Optimized Harnesses

The observed result of a benchmark run is better approximated as

$$y = f(M, H, T, B, I, \xi), \tag{11}$$

where M is the model, H the harness, T the tools, B the budget, I the infrastructure, and ξ stochasticity in users and environments. Empirical evidence already shows that differences in infrastructure, including containers, networks, and command execution, can materially change rankings on coding benchmarks (Anthropic, 2026b). We therefore recommend reporting two tracks for each model:

1. **Minimal standardized harness**: identical tools, prompts, context, budgets, retries, and environments, used to approximate controlled comparisons of model capability;
2. **Optimized disclosed harness**: model-specific optimization of planning, compression, parallelism, recovery, and tool representations is permitted, but configurations, versions, and ablation studies attributing the gains must be fully disclosed; this track compares the performance ceiling of deployable systems.

Each rollout should record at least the model snapshot, harness commit/configuration, task and environment digests, tool-schema version, seed, every retry, all actions and state diffs, verifier version, token usage, monetary cost, wall-clock time, human interventions, and final artifact. Reporting only the final answer or mean success rate cannot support failure attribution, retrospective re-evaluation, or compliance auditing.

Core finding

The value of a harness lies not in "boosting a model's benchmark score," but in turning a domain task into a resettable, interactive, verifiable, replayable, and permission-controlled experiment. General frameworks provide execution and logging, domain environments provide state and rules, and verifiers provide evidence. All three are indispensable and should not be obscured under a single term in a paper.

5 Limitations and Open Problems

Research on vertical agents is undergoing a shift in the object of evaluation: from "does the model know the domain answer?" to "can the system complete professional work in a dynamic world with the correct permissions, a reliable process, and acceptable cost?" The difficulties exposed by this shift are not merely a lack of long-horizon reasoning capacity; they are system bottlenecks jointly created by constructs, data, environments, verification, governance, and maintenance. Table 9 summarizes the research agenda that we consider most urgent.

Table 9: Open problems for the next generation of vertical agents. Priority (P) reflects our judgment based on scientific validity, deployment risk, and actionability; it is not a quantitative ranking.

Priority (P)	Open problem	Current gap	Required evidence/actionable direction
1	Construct validity	Domain knowledge, interface control, stateful workflows, and governance are conflated in a single success rate	Six-dimensional vertical profiles; orthogonal interventions on knowledge/policy/state/interface, with causal gains reported
1	Benchmark half-life	Contaminated public tasks, dependency decay, flawed tests, and live-service drift	Frozen core + refreshed shadow + sealed canary; continual review and temporal splits
1	Verifier reliability	Executability does not imply correctness; terminal-state tests miss process violations and side effects	Verifier mutation suites, equivalent solutions, forbidden actions, and human audits of false positives/negatives
1	Model–harness attribution	Retries, context, tool representations, infrastructure, and budgets jointly alter scores	Minimal standardized and optimized disclosed tracks; component ablations; unified provenance and cost curves
1	High-risk governance	A correct outcome does not establish compliant evidence collection, authorization, disclosure, or escalation	Executable policy engines, process invariants, approval and revocation, and independent audit replay
2	Synthetic-world fidelity	Scale is expanded by the same grammar or teacher, creating "false diversity"	Triangulation against real schemas, statistics, and experts; evaluate transition-graph coverage rather than task counts
2	Failure and recovery data	Demonstrations contain almost exclusively happy paths, while failed rollouts are discarded	First break points, minimal repairs, counterfactual pairs, and recovery benchmarks
2	User simulators	Simulated users are overly cooperative and stylistically homogeneous; model/seed choices alter rankings	Human-calibrated personas, adversarial/ambiguous/self-correcting users, and simulator-sensitivity reports
2	Reliability and tail risk	pass@1 conceals stochastic failures and catastrophic side effects	pass^k, worst-seed, catastrophic-rate, recovery, and selective-automation curves
2	Budget comparability	Tokens, time, monetary cost, GPU use, and human-team baselines are incommensurate	Iso-token, iso-$, iso-time, and iso-GPU-hour Pareto frontiers
3	Privacy, licensing, and representation	Medical, financial, and enterprise logs are difficult to share, while synthetic-data "realism" is validated circularly	Privacy-preserving statistics + synthetic state + blinded expert review; provenance/consent ledgers
3	Maintenance economics	Terabyte-scale images, long GPU runs, and aging websites continually raise the barrier to research	Content-addressed caches, lite/full splits, shared artifacts, and offline re-scoring

5.1 The "Vertical" Construct Remains Unstable

Many papers define a domain by a task name, website category, or professional vocabulary, without establishing whether performance derives from domain knowledge, domain policy, state management, or interface proficiency. WebArena, OSWorld, and InterCode are crucial for long-horizon control, but they primarily delimit Web, desktop, and terminal media; BFCL and ToolBench primarily test function selection and argument generation. Conversely, a static medical question-answering benchmark contains domain knowledge but lacks an agent's sequential actions and environmental feedback. Placing resources with such different scopes on a single leaderboard can lead a gain in DOM grounding to be mischaracterized as a gain in "enterprise reasoning."

An actionable direction is to turn the six-dimensional representation in Section 1.1 into an interventional evaluation: hold the task fixed and replace the domain policy; hold the tools fixed and exchange institutional state machines; hold knowledge fixed and alter the interface; or hold the terminal state fixed and add process constraints. Only when performance changes with the corresponding dimension can it support a causal claim that "the model learned domain rules." Future leaderboards should display capability profiles instead of compressing heterogeneous constructs into a single aggregate score.

5.2 The Structural Tension between Realism and Reproducibility

Real websites, SaaS products, and enterprise databases contain natural long tails, but drift as pages are updated and as localization, account state, CAPTCHAs, and third-party dependencies change. Frozen website replicas and simulated databases are reproducible, but omit production permissions, latency, exceptions, and organizational variation. WorkArena and WebArena have already introduced setup/validation code into Web evaluation, while revisions to OSWorld-Verified and τ^3 tasks likewise show that even execution-based tasks require continual auditing of both environments and graders (Xie et al., 2024; Sierra Research, 2026).

We recommend three tiers of evidence rather than a binary choice between "fully simulated" and "fully online":

1. **Frozen core**: version-locked and replayable offline, supporting method ablations and historical comparisons;
2. **Refreshed shadow set**: collected on a rolling temporal basis with sealed answers, to estimate contamination, environment drift, and generalization to new tools;
3. **Production canary**: evaluates long-tail behavior, safety, and human escalation in an authorized real system with low traffic and rollback support.

The three tiers should not be averaged into one number; they should instead be reported separately as laboratory, temporal, and deployment evidence.

5.3 The Half-Life and Contamination of Public Benchmarks

SWE-bench illustrates a typical benchmark lifecycle problem: public repositories, issues, PRs, and tests facilitate reproducibility, but also allow answers to enter pretraining, post-training, and agent trajectory corpora. In 2026, OpenAI stopped using SWE-bench Verified as a primary metric for frontier coding, citing memorization contamination and the discovery, after re-review, of numerous unstable tasks with specification or test defects (OpenAI, 2026b). This is not an idiosyncrasy of one benchmark but an inevitable pressure on any public artifact benchmark.

The absence of a benchmark name from training data is insufficient to establish freedom from contamination. Provenance audits should account for task-resolution time, the model's knowledge cutoff, repository fork/mirror similarity, patch semantics, and overlap with retrieved trajectories. Evaluation and training sets should also separate entities, policy versions, tool combinations, and workflow templates, rather than merely randomizing prompt splits. The SWE-rebench V2 strategy of continuously collecting new PRs (Badertdinov et al., 2026) should be extended to legal revisions, financial reporting seasons, SaaS product changes, and newly discovered vulnerabilities, but the refresh pipeline itself must also be versioned and subjected to sampled human review.

5.4 Synthetic Scale Does Not Imply Structural Diversity

Blueprints, personas, and parameter substitution can cheaply generate millions of trajectories, yet the samples may share a single workflow grammar, tool-dependency graph, and verifier. A random split then reduces the test set to new instantiations of training templates. "Coverage" should therefore be redefined from sample count to at least four structural measures: the number of independent state-transition rules, tool-dependency graph motifs, constraint combinations, and exception/recovery branches. Studies should also report the generator, teacher, and judge families; when all three share a lineage, generation bias can be reconfirmed during validation.

Synthetic environments require real external anchors: real schemas for structure, de-identified statistics for behavioral frequencies, and blinded expert review for rules and outcomes. Neural world models can expand rollouts inexpensively, but should not independently serve as reward oracles. Randomly sampled transitions should be compared against real programmatic environments, with transition error plotted as a function of horizon. Otherwise, small per-step errors in long trajectories can accumulate into a business world that has never existed.

5.5 From Correct Terminal States to Compliant Processes

Terminal-state verifiers are among the most important advances in existing vertical benchmarks, but "the database reached the correct state" does not establish process compliance. A financial agent may cite future information, a medical agent may bypass physician confirmation, a legal agent may use inadmissible material, and a customer-service agent may exceed its authority before rolling the action back. Conversely, checking every natural-language rationale step by step can overconstrain equivalent strategies and turn a subjective judge into the reward bottleneck.

A more robust layered design is to use hidden terminal predicates to evaluate goals, inviolable policy invariants to constrain authorization, ordering, provenance, and disclosure, state diffs to capture collateral damage, and rubric-based judging only for residual explanatory quality. During training, business invariants can be converted into dense checkpoints to aid credit assignment, while final success remains gated by the hidden terminal state. Every verifier should undergo mutation testing: alter only the output text, skip a required authorization, introduce an unrelated deletion, reuse stale evidence, or substitute an equivalent lawful path, and then check whether the grader responds as intended.

5.6 Correlated Bias and Domain Calibration in LLM Judges

Open-ended writing, research reproduction, and clinical communication cannot be fully covered by unit tests, making LLM judges unavoidable. The problem is that generators, agents, and judges often come from the same model family, allowing factual errors, format preferences, and stylistic tendencies to reinforce one another. By having authors define hierarchical rubrics, reproducing experiments in fresh containers, and evaluating the judge separately with JudgeEval, PaperBench offers a stronger design than a one-shot "overall quality from 1 to 10" score (Starace et al., 2025); nevertheless, rubric coverage, model versions, and experimental randomness must still be disclosed.

Future work should treat the judge as a model to be evaluated, not an invisible measuring instrument: disclose the prompt, model snapshot, temperature, and evidence inputs; calibrate with heterogeneous committees and human domain-expert samples; report stratum-specific agreement, confidence intervals, and bias by error type; and apply a prespecified arbitration rule when programmatic evidence conflicts with the judge. High-risk domains should also measure whether refusal or escalation is correct, rather than rewarding only confident completion.

5.7 User Simulators and Interaction-Distribution Bias

The outcome of a transactional agent task is jointly controlled by the agent and the user. The τ family has user simulators provide information, confirmations, and tool actions, an important step from static prompts toward interaction (Yao et al., 2025; Barres et al., 2025). Yet simulators are often more cooperative and consistent than real users, and are less likely to withhold information, change their minds, misunderstand, delay, or refuse. Changing the user model, seed, or turn limit can alter scores; the user simulator is therefore part of the benchmark definition, not a neutral test instrument.

A simulator calibration set is needed: human users should complete the same tasks while inhabiting specified personas, enabling comparisons of the order of information disclosure, linguistic behavior, noncooperation rates, erroneous tool actions, and mid-task intent changes. Sensitivity of agent performance to simulator family and seed should then be reported. Metrics can further isolate when to clarify, when to escalate to a human, and when to stop asking questions, preventing benchmarks from rewarding only agents that extract standard fields from compliant users.

5.8 From One-Off Success to Reliability, Recovery, and Selective Automation

Vertical deployments care not about whether a task succeeds once on average, but whether execution remains reliable across repetitions, distribution shifts, and exceptional states. τ-bench uses pass^k to reveal the rapid decline in success as repeated success is required, a measure closer to business service levels than pass@1 (Yao et al., 2025). Existing leaderboards, however, rarely report worst-seed performance, rates of catastrophic side effects, first-error locations, recovery rates, and the cost of human intervention together.

We recommend extending evaluation to risk–coverage curves: an agent may refuse or escalate low-confidence or high-risk tasks, while the system reports automated-completion coverage, conditional success, serious-error rates, and human minutes per successful task. Such selective-automation metrics better reflect practice in medicine, finance, law, and security than forcing every task to be completed automatically. Recovery should also be tested independently: after a tool timeout, partial write, permission change, user correction, or environment drift, can the agent diagnose the first break point and continue safely, rather than restarting from scratch or repeating side effects?

5.9 Resource Fairness, Sustainable Maintenance, and Open Science

Powerful environments such as MLE-bench, PaperBench, and CyberGym incur costs in GPU hours, long runtimes, and large images. Comparing a 24-hour single agent with a Kaggle leaderboard refined by human teams over many years, or allowing different models different numbers of concurrent attempts and reporting only the best result, cannot isolate model progress. Studies should plot iso-token, iso-cost, iso-wall-clock, and iso-GPU-hour curves together and disclose failed trials rather than only the single best run.

Maintenance is likewise a scientific concern. Data preparation can take days, security images can reach terabyte scale, and Web instances require continual resets. Resource releases should provide content-addressed images, layered caches, smoke/lite/full splits, environment digests, a registry of known issues, and complete trajectories that can be re-scored. If low-resource laboratories can run only a static subset, papers should explicitly state how its construct differs from the full benchmark rather than retaining the same name without qualification.

5.10 Safety, Privacy, Licensing, and Boundaries of Responsibility

Coding and security agents execute untrusted inputs; the network, filesystem, secrets, and verifier endpoints can all become shortcuts or attack surfaces. CyberGym's differential verification is strong only when the agent cannot access post-fix artifacts, reference PoCs, or hidden binaries, and when outbound connections are denied by default (Wang et al., 2026b). More generally, agents and verifiers should be separated by both privileges and containers; egress should be deny-by-default; secrets should be read-only and short-lived; write operations should use the minimum scope; output artifacts should be scanned for malicious content; and canaries should detect leakage.

Medical, financial, customer-service, and legal logs also contain private information, trade secrets, and restricted content, making the domains that most need real data the hardest to release publicly. A practical compromise is to release expert-authored simulators, de-identified statistics, and synthetic states, while independent experts calibrate populations, rules, and failure modes on blinded samples. The same model that generates the data, however, cannot be used to certify that the synthetic data are "realistic." Data cards should further record sources, consent, licenses, revocability, retention periods, and permitted training/evaluation uses.

5.11 Minimum Reporting Checklist

Table 10 presents the minimum reporting items we recommend. Its purpose is not to impose procedural overhead, but to make scores reproducible, reinterpretable, and auditable.

Core finding

The central open problem for vertical agents is not to add another collection of domain prompts, but to establish a measurement regime that can be maintained over time: constructs must be decomposable, environments resettable, data traceable, verifiers adversarially testable, harness gains attributable, risks stratified, and failed trajectories replayable under revised rules. Only then can benchmark progress translate into deployable professional reliability.

Table 10: Minimum reporting checklist for vertical-agent papers and leaderboards.

Object	Required disclosures	At least one diagnostic output
Task / data	Source, date, license, entity/policy/environment split, and contamination audit	Template/transition-graph coverage and failure-type distribution
Model	Provider snapshot, system prompt, temperature, and context strategy	Ablations removing memory/planning/retries
Tools / policy	Schema/version, authorization scope, visible knowledge, and approval rules	Tool-level success, unauthorized/irrelevant calls, and retrieval freshness
Environment	Image/state digest, reset procedure, seed, network, and dependencies	Reset hash, drift rate, and number of failed environments
Verifier	Version, hidden status, program/metric/judge composition, and arbitration rule	Mutation tests, false positives/negatives, and agreement with humans
Budget	Tokens, cost, wall time, GPUs, concurrency, attempts, and human minutes	Cost–success Pareto curve and complete trial distribution
Outcome	pass@1, pass^k, progress, policy, side effects, and refusal/escalation	Worst-seed, catastrophic rate, and recovery rate
Provenance	Model/harness/task/environment/verifier commits, full traces, and artifacts	Replayable/re-scorable examples and known-issue registry

6 Conclusion

This survey defines a vertical agent through a "domain closed loop" rather than a task label: a model policy must form a system together with domain tasks, knowledge and data, tools, a persistent environment, verifiers, and governance constraints. This definition explains why static domain question answering, horizontal function calling, Web/desktop control, and real-world industry workflows cannot be placed directly on a single capability axis. It also explains why the same foundation model can exhibit substantial score differences under different harnesses, budgets, and infrastructures.

Our synthesis of four bodies of literature yields four conclusions. First, the most informative benchmarks are not merely "harder"; they provide resettable state, executable tools, strong terminal-state evidence, and process constraints. The relatively rapid progress in software engineering, transactional customer service, and cybersecurity has coincided with—and been facilitated by—the availability of tests, state diffs, and differential oracles. Second, the basic unit of vertical data has evolved from a document or successful conversation into an executable task package, whose scarcest asset is a failed trajectory annotated with its first error location, cause, minimal repair, and counterfactual. Third, the harness is part of the system under evaluation: general experimentation frameworks, interface environments, domain worlds, and graders solve different problems, and fair comparison requires separate minimal standardized and optimized disclosed tracks. Fourth, our synthesis of public high-risk evaluations indicates that the main challenge has expanded from "answering like an expert" to evidence freshness, state fidelity, permissions, process compliance, calibration, refusal, and human escalation.

The central research object for the next generation of vertical agents is therefore neither a longer prompt nor a larger benchmark, but a continually maintainable domain world. Such a world has rules and states grounded in reality; can be safely reset and interacted with; prioritizes programmatic evidence when verifying outcomes; records the process in full; and supports replay after policies, environments, or models change. Only when model improvements can be attributed, reproduced, and risk-constrained within this closed loop does "vertical capability" become verifiable professional competence rather than a leaderboard label.

Design rule

Build new verticals in this order: define terminal states and forbidden side effects; establish resettable state and least-privilege tools; add hidden verifiers and audit logs; then scale tasks and trajectories. Otherwise, synthetic data only scales unmeasured bias.

References

Anthropic. 2024. Introducing the model context protocol. Anthropic News. Accessed 2026-08-19.

Anthropic. 2026a. Claude Opus 4.8. Anthropic model page. Accessed 2026-08-19.

Anthropic. 2026b. Quantifying infrastructure noise in agentic coding evals. Anthropic Engineering. Accessed 2026-08-19.

Rahul K. Arora and 1 others. 2025. HealthBench: Evaluating large language models towards improved human health. *Preprint*, arXiv:2505.08775.

Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, and Alexander Golubev. 2026. SWE-rebench V2: Language-agnostic SWE task collection at scale. *Preprint*, arXiv:2602.23866.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-Bench: Evaluating conversational agents in a dual-control environment. *Preprint*, arXiv:2506.07982.

Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. 2025. Finance agent benchmark: Benchmarking LLMs on real-world financial research tasks. *Preprint*, arXiv:2508.00828.

Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier de Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. 2024. WorkArena++: Towards compositional planning and reasoning-based common knowledge work tasks. *Preprint*, arXiv:2407.05291.

Shihao Cai, Runnan Fang, Jialong Wu, Baixuan Li, Xinyu Wang, Yong Jiang, Liangcai Su, Liwen Zhang, Wenbiao Yin, Zhen Zhang, Fuli Feng, Pengjun Xie, and Xiaobin Wang. 2025. AutoForge: Automated environment synthesis for agentic reinforcement learning. *Preprint*, arXiv:2512.22857.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. 2025. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*.

Aili Chen, Chi Zhang, Junteng Liu, Jiangjie Chen, Chengyu Du, Yunji Li, Ming Zhong, Qin Wang, Zhengmao Zhu, Jiayuan Song, Ke Ji, Junxian He, Pengyu Zhao, and Yanghua Xiao. 2026a. DIVE: Scaling diversity in agentic task synthesis for generalizable tool use. *Preprint*, arXiv:2603.11076.

Haolin Chen and 1 others. 2026b. χ-Bench: Benchmarking healthcare agents in policy-dense operational workflows. *Preprint*, arXiv:2605.16679.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, and 1 others. 2025. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *Preprint*, arXiv:2509.16941.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, volume 36, pages 28091–28114.

Guanting Dong, Xiaoshuai Song, Yuyang Hu, Jiajie Jin, Chenghao Zhang, Yifei Chen, Xiaoxi Li, Huaying Yuan, Xinyu Yang, Tongyu Wen, Jiejun Tan, Hongjin Qian, Shijue Huang, Junting Lu, Zhenyu Li, Wanjun Zhong, Yutao Zhu, Tat-Seng Chua, Zhicheng Dou, and Ji-Rong Wen. 2026. Towards long-horizon agents: A survey.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, and 1 others. 2024. WorkArena: How capable are web agents at solving common knowledge work tasks? In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 11642–11662.

Runnan Fang, Shihao Cai, Baixuan Li, Jialong Wu, Guangyu Li, Wenbiao Yin, Xinyu Wang, Xiaobin Wang, Liangcai Su, Zhen Zhang, and 1 others. 2025. AgentScaler: Towards general agentic intelligence via environment scaling. *Preprint*, arXiv:2509.13311.

Google DeepMind. 2026. Gemini 3.5 Flash model card. Google DeepMind model card. Accessed 2026-08-19.

Lianghong Guo, Yanlin Wang, Caihua Li, Wei Tao, Pengyu Yang, Jiachi Chen, Haoyu Song, Duyu Tang, and Zibin Zheng. 2025. SWE-Factory: Your automated factory for issue resolution training data and evaluation benchmarks. *Preprint*, arXiv:2506.10954.

Harbor Framework Contributors. 2026. Harbor: A framework for running and evaluating agents. Software repository and documentation. Accessed 2026-08-19.

Harvey. 2026. Introducing harvey's legal agent benchmark. Harvey Blog. Accessed 2026-08-19.

Yixing Jiang, Kameron C. Black, Gloria Geng, Danny Park, James Zou, Andrew Y. Ng, and Jonathan H. Chen. 2025. MedAgentBench: A virtual EHR environment to benchmark medical LLM agents. *NEJM AI*.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*.

Eric Y. Kim and Jie Huang. 2026. FinRetrieval: A benchmark for financial data retrieval by AI agents. *Preprint*, arXiv:2603.04403.

Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, and 1 others. 2025. The BrowserGym ecosystem for web agent research. *Transactions on Machine Learning Research*.

Haitao Li, Junjie Chen, Jingli Yang, Qingyao Ai, Wei Jia, Youfeng Liu, Kai Lin, Yueyue Wu, Guozhi Yuan, Yiran Hu, Wuyue Wang, Yiqun Liu, and Minlie Huang. 2025. LegalAgentBench: Evaluating LLM agents in legal domain. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*.

Qianchu Liu and 1 others. 2026. HealthAgentBench: Benchmarking end-to-end healthcare agents across diverse clinical workflows. *Preprint*, arXiv:2606.31179.

Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, and 1 others. 2024. APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Preprint*, arXiv:2406.18518.

Jiaxuan Lu, Kong Wang, Yemin Wang, Qingmei Tang, Hongwei Zeng, Xiang Chen, Jiahao Pi, Shujian Deng, Lingzhi Chen, Yi Fu, Kehua Yang, and Xiao Sun. 2026. FinToolBench: Evaluating LLM agents for real-world financial tool use. *Preprint*, arXiv:2603.08262.

Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024. WebLINX: Real-world website navigation with multi-turn dialogue. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 33007–33056.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, and 1 others. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *Preprint*, arXiv:2601.11868.

Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, and 1 others. 2025. MLGym: A new framework and benchmark for advancing AI research agents. In *Conference on Language Modeling*.

OpenAI. 2025. GPT-5 system card. OpenAI system card. Accessed 2026-08-19.

OpenAI. 2026a. Separating signal from noise in coding evaluations. OpenAI technical audit. Accessed 2026-08-19.

OpenAI. 2026b. Why we no longer evaluate SWE-bench Verified. OpenAI technical audit. Accessed 2026-08-19.

OpenAI and 1 others. 2025. GDPval: Evaluating AI model performance on real-world economically valuable tasks. *Preprint*, arXiv:2510.04374.

Akshara Prabhakar, Zuxin Liu, Ming Zhu, Jianguo Zhang, Tulika Awalgaonkar, Shiyu Wang, Zhiwei Liu, Haolin Chen, Thai Hoang, Juan Carlos Niebles, Shelby Heinecke, Weiran Yao, Huan Wang, Silvio Savarese, and Caiming Xiong. 2025. APIGen-MT: Agentic pipeline for multi-turn data generation via simulated agent-human interplay. *Preprint*, arXiv:2504.03601.

Salesforce Research. 2025. CRMArena-Pro: Holistic assessment of LLM agents across diverse business scenarios and interaction dynamics. *Preprint*, arXiv:2505.18878.

Samuel Schmidgall, Rojin Ziaei, Carl Harris, Eduardo Reis, Jeffrey Jopling, and Michael Moor. 2026. AgentClinic: A multimodal benchmark for tool-using clinical AI agents. *npj Digital Medicine*.

Sierra Research. 2026. τ^3-bench. Benchmark website. Accessed 2026-08-19.

Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. 2026. EnvScaler: Scaling tool-interactive environments for LLM agent via programmatic synthesis. *Preprint*, arXiv:2601.05808.

Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. 2025. PaperBench: Evaluating AI's ability to replicate AI research. *Preprint*, arXiv:2504.01848.

Xiaoyu Tian, Haotian Wang, Shuaiting Chen, Hao Zhou, Kaichi Yu, Yudian Zhang, Jade Ouyang, Junxi Yin, Jiong Chen, Baoyan Guo, Lei Zhang, Junjie Tao, Yuansheng Song, Ming Cui, and Chengwei Liu. 2026. ASTRA: Automated synthesis of agentic trajectories and reinforcement arenas. *Preprint*, arXiv:2601.21558.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 16022–16076.

UK AI Security Institute. 2024. Inspect AI: Framework for large language model evaluations. Project documentation. Accessed 2026-08-19.

Bertie Vidgen, Austin Mann, Abby Fennelly, John Wright Stanly, Lucas Rothman, Marco Burstein, Julien Benchek, David Ostrofsky, Anirudh Ravichandran, Debnil Sur, and 1 others. 2026. APEX-Agents. *Preprint*, arXiv:2601.14242.

Alex Wang, Georg Meinhardt, Jacob Katz, Joseph H. Kim, Pratyush K. Chaudhary, Chase Blagden, and Eric Xu. 2026a. BigFinanceBench: A workflow-grounded benchmark for financial-research agents. *Preprint*, arXiv:2606.03829.

Zhun Wang, Tianneng Shi, Jingxuan He, Matthew Cai, Jialin Zhang, and Dawn Song. 2026b. CyberGym: Evaluating AI agents' cybersecurity capabilities with real-world vulnerabilities at scale. In *International Conference on Learning Representations*.

Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, and 1 others. 2025. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 66772–66832.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, and 1 others. 2024. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, volume 37, pages 52040–52094.

Zhangchen Xu, Adriana Meza Soria, Shawn Tan, Anurag Roy, Ashish Sunil Agrawal, Radha Poovendran, and Rameswar Panda. 2025. TOUCAN: Synthesizing 1.5m tool-agentic data from real-world MCP environments. *Preprint*, arXiv:2510.01179.

John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025. SWE-smith: Scaling data for software engineering agents. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2023. InterCode: Standardizing and benchmarking interactive coding with execution feedback. *Preprint*, arXiv:2306.14898.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*.

Ailing Yu and 1 others. 2025. MedReseacher-R1: Expert-level medical deep researcher via a knowledge-informed trajectory synthesis framework. *Preprint*, arXiv:2508.14880.

Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Jasper, and 1 others. 2025. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *International Conference on Learning Representations*.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, and 1 others. 2024. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*.

Table 11: Primary notation used in the main text. Domain content, runtime implementation, task records, and experimental configurations use distinct symbols to avoid recursively nested notation.

Symbol	Meaning	Symbol	Meaning
Ω_d	Six-dimensional domain specification: tasks, knowledge, tools, environment, verifier, and governance	$\mathcal{H}_d^{(m)}$	Harness implementing the domain specification in evaluation, training, or deployment mode m
$\mathcal{I}_d$	Test tasks and initial states instantiated from the domain specification	$\mathcal{Q}$	Experimental configuration, including budget, retries, sampling, human intervention, and infrastructure
$\mathcal{S}_i$	Required executable task specification	$\mathcal{R}_i$	Optional set of successful, failed, recovery, and counterfactual rollouts
G_i	Conjunction of four programmatic hard gates for schema, transition, terminal state, and safety	J_i	Open-ended semantic residual score, reported only when applicable
P_d	State-transition kernel of the domain environment	r_i	Per-run reward obtained from the programmatic hard gates and optional semantic residual

A Survey Methodology and Resource-Coding Protocol

We conduct a *question-driven scoping review* and maintainable resource inventory rather than claiming an exhaustive systematic review. The evidence was frozen on 2026-08-18; the main text and CSV files reflect papers, project pages, code repositories, data cards, model system cards, and technical reports from research institutions that were publicly verifiable by that date. Because agent resources evolve rapidly, multiple versions often share a name, and some environments require accounts or proprietary software, we do not provide a PRISMA flowchart or present counts of search results, deduplicated records, or exclusions as unverified "systematic-review counts."

A.1 Retrieval Questions and Source Priorities

Resource collection was organized around four questions: Which tasks embody real domain rules and a stateful closed loop? How does training-data construction generate tasks, trajectories, and verifiers? Which harness components make execution and verification reproducible? What remains missing between public evaluation and governed deployment? Evidence sources were prioritized in the following order:

1. Papers, official data cards, official code repositories, and benchmark project pages;
2. Model providers' system cards and research pages, used to identify the evaluations actually applied to models such as Claude, GPT, and Gemini, without treating vendor-reported scores as directly comparable under a common protocol;
3. Author talks, technical blogs, or leaderboards, used only to supplement version, harness, and run-configuration details and never to replace primary papers or code;
4. Secondary surveys, used to discover candidates but not as the sole source for key numbers, years, or methodological conclusions.

Each machine-readable record retains one `primary_url`. We prefer links to papers, official repositories, or official research pages over aggregators. The `year` field denotes the year in which the first version became public, which may differ from the year of formal conference publication. Citations in the main text and entries in the machine-readable tables serve different purposes: the former support arguments, while the latter support retrieval, filtering, and continual updates.

A.2 Primary Notation

A.3 Inclusion, Downgrading, and Exclusion Rules

A candidate resource enters the core inventory if it satisfies at least one of the following criteria: (i) an agent executes sequential actions in an observable environment and can be evaluated with tests, state

Table 12: Resource scope categories. The scope category answers "where is the resource vertical?"; the evidence level answers "how far does the closed loop extend?"

Category	Minimum criterion	Reporting treatment
Industry vertical	Jointly includes domain rules or professional constraints, domain state/deliverables, and a domain verifier	Include in the core domain table and report V level, environment, verifier, policy, and primary limitations
Functional vertical	Forms a closed loop around a professional function such as software engineering, ML engineering, terminal operations, or research reproduction, without being tied to one industry	May serve as core evidence for professional work; must not be extrapolated to policies in every industry
Interface domain	Primarily addresses browser, desktop, mobile, terminal, or cross-application operation	List separately as environment and interaction infrastructure; UI success cannot substitute for industry correctness
Horizontal substrate	Provides function calling, tool data, protocols, general-purpose evaluation, or observability	Use for cold starts and infrastructure comparisons; do not merge its rankings with domain benchmarks
Boundary evidence	Presents a realistic professional scenario or expert delivery standard but lacks an executable stateful closed loop	Interpret as V0–V2 and state what capability it can and cannot establish

diffs, or task metrics; (ii) the task explicitly encodes the knowledge, policies, tools, or delivery standards of an industry or professional function; or (iii) the resource provides a reusable data-construction pipeline, environment runtime, control loop, verifier, or observation interface. Resources limited to static question answering, single-turn classification, or general-purpose function calling may still be retained as boundary evidence or horizontal substrates, but they are not ranked together with V3 benchmarks in the industry-vertical category.

Resources are interpreted with reduced evidentiary weight or temporarily excluded when they lack a verifiable primary link; report promotional scores without explaining tasks or graders; do not permit contributions from the model, harness, and external tools to be distinguished; depend on undisclosed private data without sufficient methodological detail; or include "agent" in their name while testing only static domain knowledge. Closed benchmarks may enter the boundary discussion, but their reproducibility and auditability must be marked explicitly.

A.4 Five Scope Categories and Five Evidence Levels

To avoid treating any appearance of domain terminology as evidence of verticality, the CSV files use `scope_class` to encode the nature of each resource:

Evidence strength follows the cumulative V0–V4 ladder in Table 1. V0 is a static prerequisite capability. V1 adds real or representative scenarios and task-relevant delivery standards; V2 further adds traceable data sources, tools, or interface actions; V3 additionally requires resettable initial states, state changes, and executable terminal-state verification; and V4 also requires real permissions, human escalation, rollback, auditing, and production monitoring. The cumulative criteria are interpreted relative to scope: industry and functional verticals use professional standards, whereas interface domains use explicit task-completion conditions and traceable operations, but cannot thereby claim industry correctness. A level characterizes *public evidence*, not a system's latent capability. HealthBench, for example, has strong physician-authored rubrics but no EHR state writes and therefore remains V1, whereas MedAgentBench reaches V3 through its FHIR environment and state assertions.

A.5 Fields, Review, and Update Protocol

All three CSV files are standard comma-separated files encoded in UTF-8, with field names on the first row and every field enclosed in double quotation marks. `benchmarks.csv` records the domain, year, scope category, V level, role, environment, verifier, primary link, and main limitation. `datasets.csv` additionally distinguishes the source unit from the synthesis or collection method. `harnesses.csv`

encodes the harness role, primary layer, runtime, verifier support, and observability.

Adding or updating a resource requires at least four checks: confirm the primary link and first-public-version year; determine from public methods or code whether the environment is resettable; describe model judges separately from programmatic verifiers; and check whether a new version has advanced the resource from V1/V2 to V3. If a benchmark changes only the model without altering tasks, the environment, or the grader, no new resource row is created. If the task distribution, environment, or verifier changes materially, the revision is recorded as a new version row and the prior version is retained, preserving the semantics of historical scores.

Core finding

The purpose of this inventory is not to create the longest possible list, but to make every resource answerable to the same questions: Does it evaluate an industry, a function, or an interface? Can its initial state be reset? Who verifies success, and on what basis? Are failures, side effects, and budgets visible? A popular entry that cannot answer these questions should remain a candidate resource rather than strong evidence.

B Resource Inventories: Selected Views and Complete Machine-Readable Collections

This appendix presents selected views of the three machine-readable tables. The frozen release contains 39 benchmarks, 23 data assets/synthesis pipelines, and 17 harness/protocol components. Complete fields, primary links, and resource-specific limitations appear in `data/benchmarks.csv`, `data/datasets.csv`, and `data/harnesses.csv`, respectively. The tables below provide a compact map of the research landscape; they do not replace the more detailed environment, verification, and provenance information in the CSV files.

B.1 Benchmarks: Ordered by Closed-Loop Characteristics, Not Industry Labels

Table 13: Selected view of representative benchmarks. V denotes the level of public evidence, not a safety certification.

Resource	Domain/scope	V	Environment	Verifier	Primary interpretive boundary
SWE-bench	Software engineering; functional vertical	3	Real repositories and terminal containers	fail-to-pass + regression tests	Contamination, underspecified tests, and environment configuration can distort scores
SWE-bench Pro	Software engineering; functional vertical	3	Longer-horizon tasks in real repositories	Task-specific and regression tests	Higher execution cost; public tasks still have a short half-life
Terminal-Bench 2.0	Terminal work; functional vertical	3	Isolated CLI containers	Per-task unit/integration tests	Sensitive to resources and network access; not an industry-rule closed loop
CyberGym	Cybersecurity; industry vertical	3	Repositories before and after vulnerability fixes	Differential crash oracle	High-risk execution and sandboxing are costly; a crash does not represent a complete compliant path
Finance Agent	Financial research; industry vertical	2	Web, EDGAR, and computational tools	Expert answers, atomic rubrics, and citations	Does not cover trading, approval, or regulatory state machines
MedAgentBench	Medical EHR; industry vertical	3	Virtual FHIR EHR	Patient resources and terminal-state assertions	Simulated records do not generalize to hospital permissions or patient outcomes

Continued on next page

Table 13 (continued)

Resource	Domain/scope	V	Environment	Verifier	Primary interpretive boundary
AgentClinic	Clinical diagnosis and treatment; industry vertical	3	Patients, examinations, and multimodal tools	Diagnosis and patient-centered metrics	The patient simulator itself becomes an object of measurement
HealthBench	Medical communication; boundary evidence	1	Multi-turn health dialogues	Physician rubrics + calibrated model judge	Has no EHR state and cannot serve as evidence for a medical execution agent
LegalAgentBench	Law; industry vertical	2	Legal corpus and 37 tools	Keyword answers and process rate	Keywords cannot substitute for legal validity or equivalent arguments
Harvey LAB	Law; boundary evidence	1	Closed client matters and files	Lawyer-authored atomic rubrics, all-pass	Strong deliverables but weak programmatic state; run specifications continue to evolve
MLE-bench	ML engineering; functional vertical	3	Kaggle competitions and GPUs	Competition metrics and medal thresholds	Compute and historical priors confound human–agent comparisons
PaperBench	Research reproduction; functional vertical	3	Paper containers and fresh runs	Hierarchical outcomes + JudgeEval	Reproduction is not open-ended research; review and reruns are expensive
RE-Bench	AI R&D; functional vertical	3	Seven R&D environments	Continuous scores and time-matched humans	Few tasks; short-duration performance does not generalize to multi-day research
τ / τ^2 / τ^3-bench	Customer service, knowledge, and voice; industry vertical	3	DB, policy, tools, knowledge, and user simulation	Terminal-state, retrieval, and policy assertions; pass^k	User/voice models and task versions must be fixed
WorkArena++	Enterprise knowledge work; functional vertical	3	ServiceNow and composite tasks	setup/validate + oracle traces	A single platform and hand-authored workflow grammar
CRMArena-Pro	CRM; industry vertical	3	CRM state, personas, and multi-turn interaction	State queries, success, and confidentiality checks	Simulated organizations and real permissions remain mismatched
WebArena / OSWorld	Browser/desktop; interface domain	3	Self-hosted websites or virtual machines	DOM, files, DB, and application state	Establishes interface control, not correctness under industry rules and policies
BFCL / ToolSandbox	Functions and tools; horizontal substrate	1–3	Schemas or simulated stateful tools	AST, execution, milestones, and terminal state	General-purpose tool competence cannot replace a domain state machine
GDPval	Economic work; boundary evidence	1	Professional files and deliverables	Expert preference/reference comparisons	One-off delivery is not a multi-day closed loop or process compliance

B.2 Data Assets and Synthesis Pipelines: From Real Seeds to Environment-First Construction

Table 14: Representative data assets and synthesis pipelines. Horizontal substrates are listed as components for bootstrapping verticals, not as endpoints of vertical development.

Resource	Role/scope	Construction logic	Validation	Primary gap
SWE-bench corpus	Real artifacts; functional vertical	Pairs issues, base commits, PRs, and repository environments	fail-to-pass / pass-to-pass	Public-patch contamination; covers only reconstructable historical records
SWE-smith	Programmatic synthesis; functional vertical	Builds repository environments and injects mutations that cause tests to fail	Test differentials and environment execution	Synthetic bugs tend to be local and are constrained by existing test coverage
SWE-rebench V2	Continual refresh; functional vertical	Collects new multilingual PRs, generates setup procedures, and filters by execution	Test differentials + judge ensemble	Temporal drift reduces comparability across versions
CyberGym	Real artifacts; industry vertical	Packages historical vulnerabilities, vulnerable and fixed versions, and PoCs	Differential pre-/post-patch crashes	High-risk code, licensing, and build cost
PaperBench	Expert decomposition; functional vertical	Authors decompose reproduction goals into hierarchical outcomes	JudgeEval + fresh reproduction	Expert time and independent reruns are expensive
Mind2Web / WebLINX	Human demonstrations; interface domain	Records actions on real websites, DOM state, screenshots, and collaborative dialogue	Offline action/element metrics	Mostly happy paths; no resettable online terminal state
τ^2 task blueprints	Policy/specification blueprints; industry vertical	Composes tasks from policies, initial states, dual-control actions, and goals	Terminal-state, communication, and policy assertions	Combinatorial count does not imply real structural diversity
WorkArena++ generator	Workflow blueprints; functional vertical	Composes atomic tasks while jointly generating setup, oracle, and validation	Programmatic state validation	A single SaaS platform and hand-authored grammar
AgentTrek	Tutorial-guided replay; interface domain	Extracts tasks from Web tutorials and guides trajectory replay	Replay consistency and visual/model critics	Bias toward canonical paths; latent business errors are difficult to detect
APIGen / APIGen-MT	Bootstrapping; horizontal substrate	Generates single- and multi-turn tasks and action blueprints from API schemas	Schema, execution, and semantic/reviewer checks	Lacks domain policies, permissions, and state machines
TOUCAN	MCP trajectories; horizontal substrate	Scales trajectories from real MCP servers and tools	Schema, execution, and consistency filtering	Service availability and supply-chain risks vary across deployments
DIVE	Evidence-first; horizontal substrate	Executes tools to obtain evidence, then infers tasks entailed by that evidence	Evidence, execution, and structural diversity	Tool evidence does not provide institutional rules or high-risk consequences
EnvScaler / ASTRA	Environment-first; horizontal substrate	Programmatically generates state machines, scenarios, SFT trajectories, and RL arenas	Rule functions and tool execution	Self-consistent rules may not be realistic; verifiers may be exploitable
AutoForge / AgentScaler	Environment scaling; horizontal substrate	Builds environments from tool documentation, learning general primitives before specialization	Programmatic rules and success functions	Requires external grounding in real schemas, behavioral statistics, and experts
AppWorld packages	Environment-first; interface domain	Builds a controllable application world before generating tasks and state tests	Goal states + forbidden side effects	Limited coverage of production authentication, exceptions, and API drift

B.3 Harnesses: Frameworks, Environments, Graders, and Protocols Are Distinct Layers

Table 15: Representative harnesses and infrastructure. A comparison should first establish whether a resource is a general framework, domain environment, native grader, interoperability protocol, or observability component.

Resource	Role/scope	Primary layer	Verifier/observation	What it cannot replace
Inspect AI	General evaluation framework; horizontal substrate	Task/solver/tool contracts, sandbox interfaces, scorers, and logs	Custom programmatic/model scorers; sample-level events and costs	Does not supply domain state, policy, or strong verifiers
Harbor	General evaluation orchestration; horizontal substrate	Containerized tasks, agent/benchmark adapters, concurrency, and result normalization	Reuses native graders; records resources and artifacts	Cannot eliminate differences in underlying environments and graders
BrowserGym	Browser environment; interface domain	Playwright, observation/action spaces, and benchmark interfaces	Actions, DOM, screenshots, and Web state	Does not supply industry rules, permissions, or governance
AgentLab	Web experimentation framework; interface domain	Agent control loops, configuration, execution, and aggregation	Trajectories, screenshots, errors, and outcomes	Gains from an optimized scaffold cannot be attributed directly to the model
SWE-bench harness	Software evaluator; functional vertical	Repository images, patches, test execution, and reports	Test outputs, patches, and container logs	Test defects, contamination, and resource confounds still require auditing
τ family runner	Customer service, knowledge, and voice; industry vertical	DB, policy, tool/knowledge/voice loops, and user simulator	Terminal state, retrieval, policy, dialogue, and pass^k	Simulators and task versions cannot replace the real user distribution
WorkArena++	Enterprise Web environment; functional vertical	ServiceNow setup, BrowserGym, composite tasks, and validation	State validation, oracle traces, and browser trajectories	A single platform cannot represent all enterprise systems
AppWorld	Multi-application environment; interface domain	API contracts, resettable DB, and cross-application control loops	State diffs, side-effect tests, and API trajectories	Simulated applications cannot replace production permissions and API governance
CyberGym	Security environment; industry vertical	Vulnerability images, restricted terminals, PoC execution, and differential verification	Builds, commands, crashes, and artifact records	A crash oracle cannot replace attack-path and egress governance
MLE-bench / MLGym	ML engineering environment; functional vertical	Data, GPUs, code experiments, submissions, or continuous rewards	Metrics, code, experiment logs, and resources	Metrics alone cannot establish research novelty or production quality
PaperBench	Research-reproduction grader; functional vertical	Containers, deliverables, hierarchical rubrics, and independent reruns	JudgeEval, commands, and fresh reproduction	Model judges still require meta-evaluation; cost impedes multiple seeds
OSWorld / InterCode	Desktop/terminal; interface domain	VM or container reset, action interfaces, and execution evaluators	Screens/output, files, DB, and task state	Interface success does not establish domain compliance

Continued on next page

Table 15 (continued)

Resource	Role/scope	Primary layer	Verifier/observation	What it cannot replace
MCP	Tool protocol; horizontal substrate	Client–server contracts, tool/resource exposure, and capability negotiation	Protocol messages can be logged; no built-in success criterion	Not a sandbox, trust model, audit system, or domain verifier
OpenTelemetry	Observability standard; horizontal substrate	Traces, metrics, logs, collectors, and cross-service correlation	Spans, errors, costs, and verifier events	Cannot automatically generate business semantics, privacy policies, or correctness judgments

B.4 Maintenance Recommendations

The machine-readable tables can serve as data sources for future websites, retrieval interfaces, or experiment registries. Updates should retain rows for prior versions and add `version`, `environment_digest`, `verifier_commit`, `license`, and `last_checked` fields rather than overwriting historical records. A leaderboard should also maintain a separate run table whose foreign key references the resource version and whose fields record the model snapshot, harness commit, budget, seed, hardware, outcome, and trajectory location. This separation keeps "what the resource is" distinct from "how a system scored on a particular run" and prevents the benchmark inventory from being misused as a static model leaderboard.